

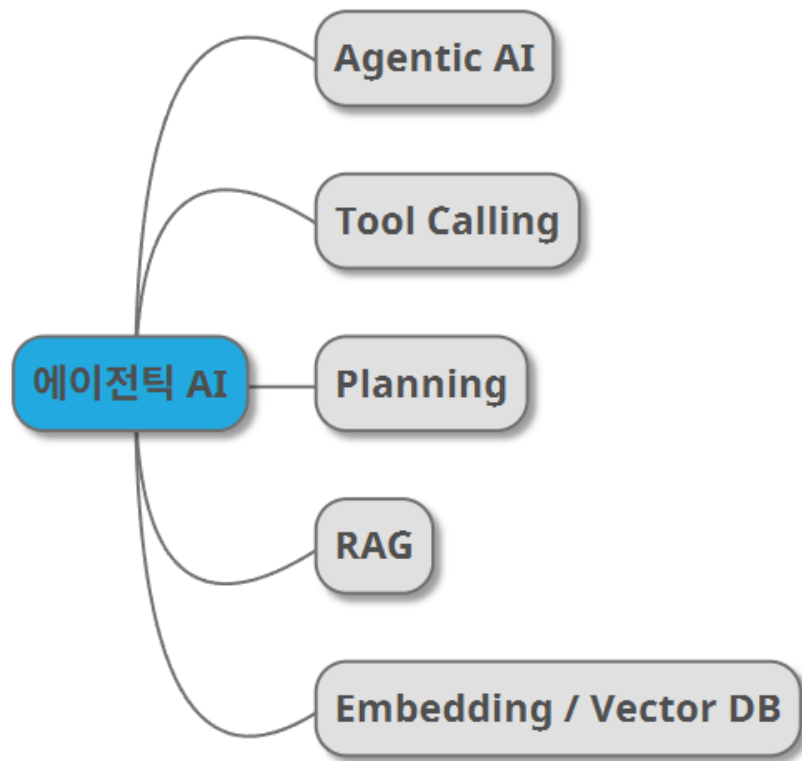


충북대학교 전자정보대학
소프트웨어학부

에이전틱 AI : Agents, Prompt Engineering

이건명

충북대학교 소프트웨어학부

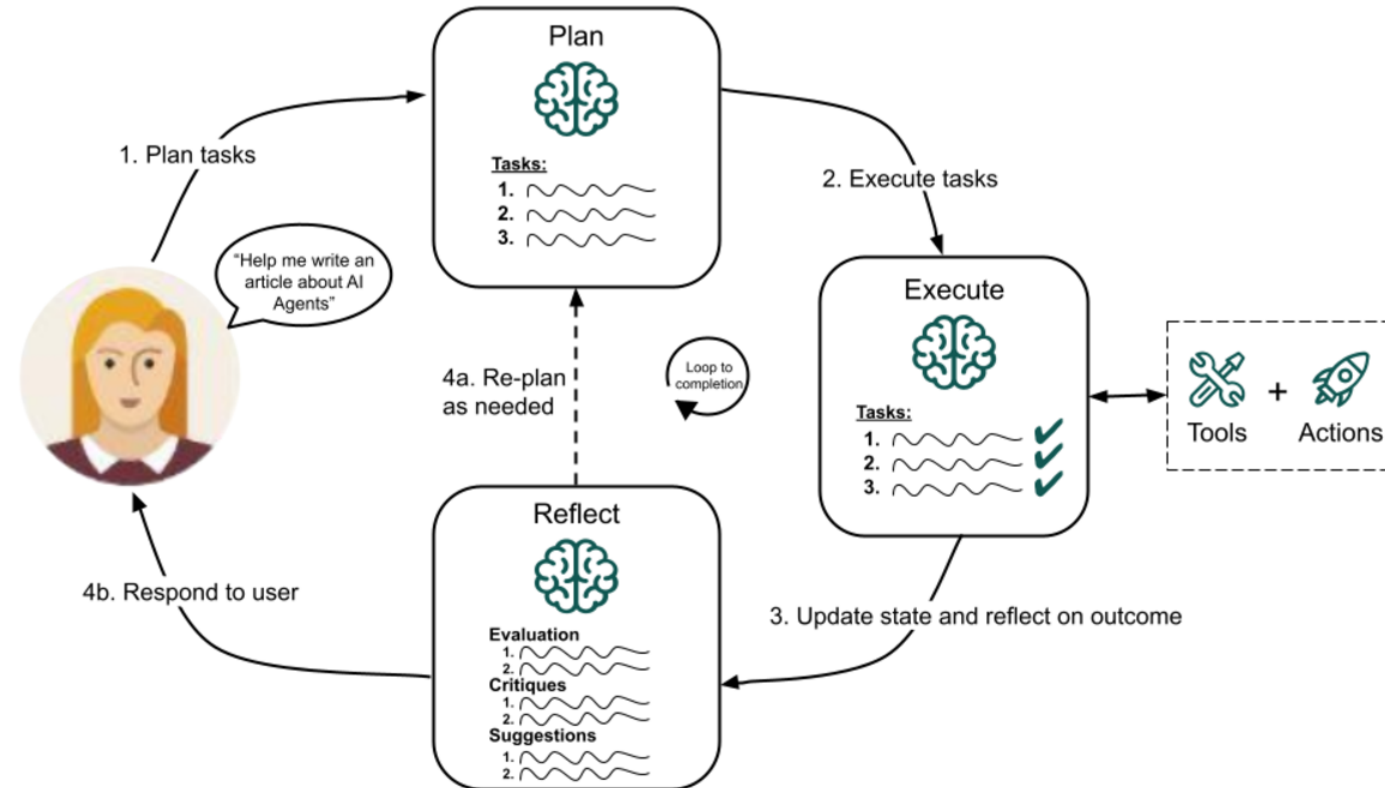
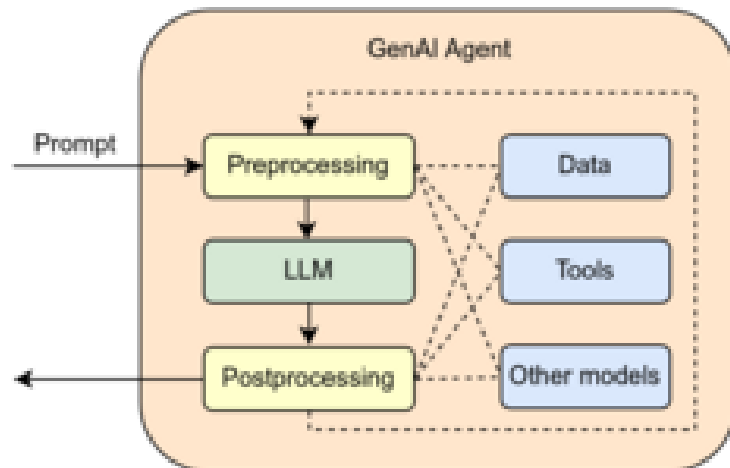


- 
- 에이전트
 - 프롬프트 엔지니어링(Prompt Engineering)
 - 컨텍스트 엔지니어링(Context Engineering)

1. 에이전트

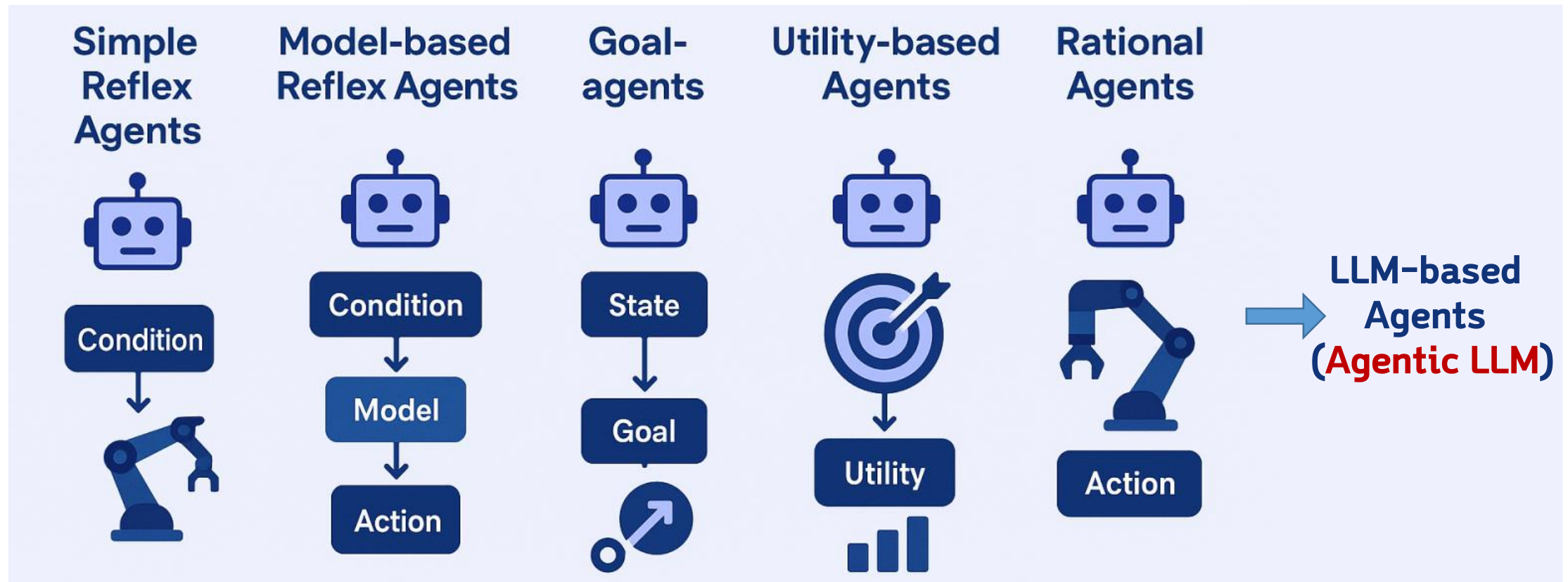
❖ AI 에이전트(agent)

- 환경을 인식(Perceive)하고, 계획(Plan)을 세우며, 도구(Tool)를 활용해 자율적으로 목표를 달성하는 소프트웨어 시스템
- 맡겨진 일을 자율적으로 처리하는 인공지능
- LLM을 Brain 역할에 활용
- 도구(외부 프로그램) 사용
- 문제해결을 위한 계획 수립(planning)
- 에이전트 간의 협력
 - 다중 에이전트 시스템



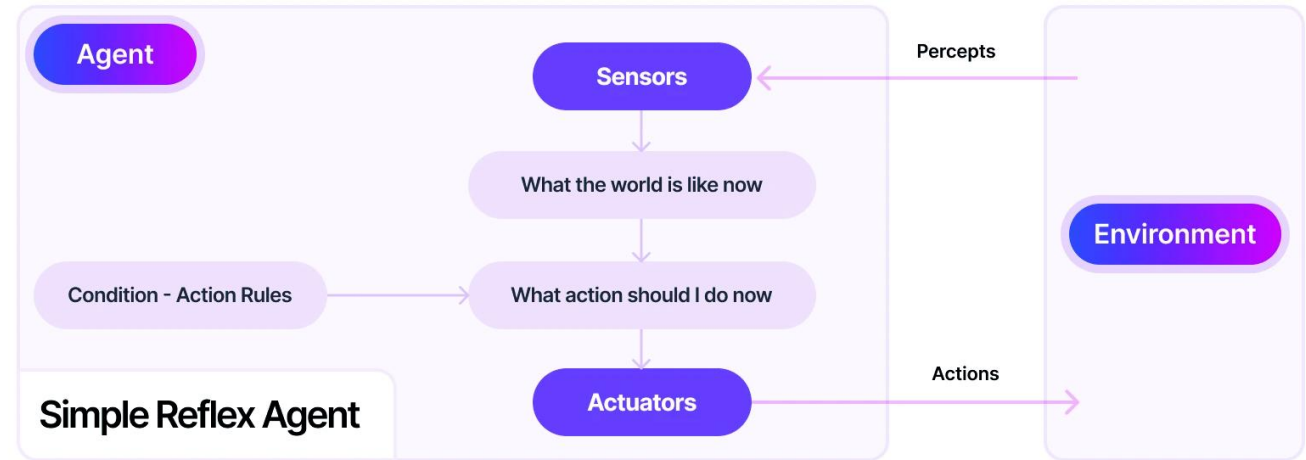
1.1 에이전트의 형태

❖ 에이전트의 진화

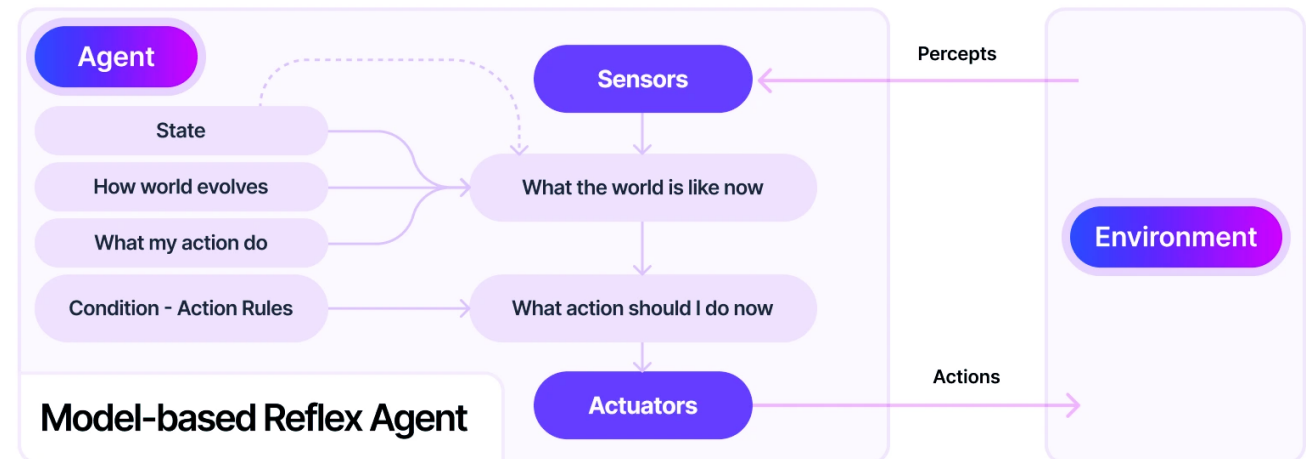


에이전트의 형태

❖ Simple Reflex Agent

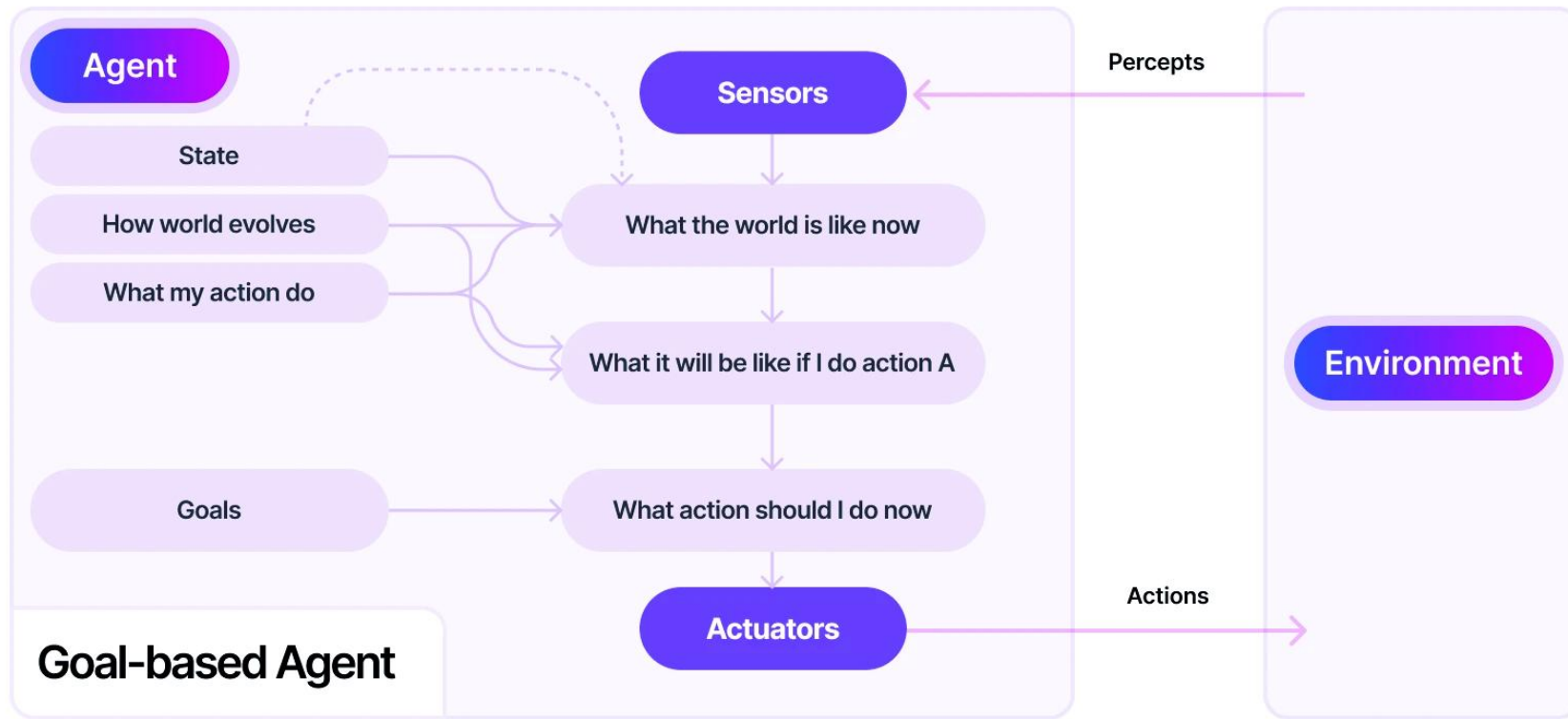


❖ Model-based Reflex Agent



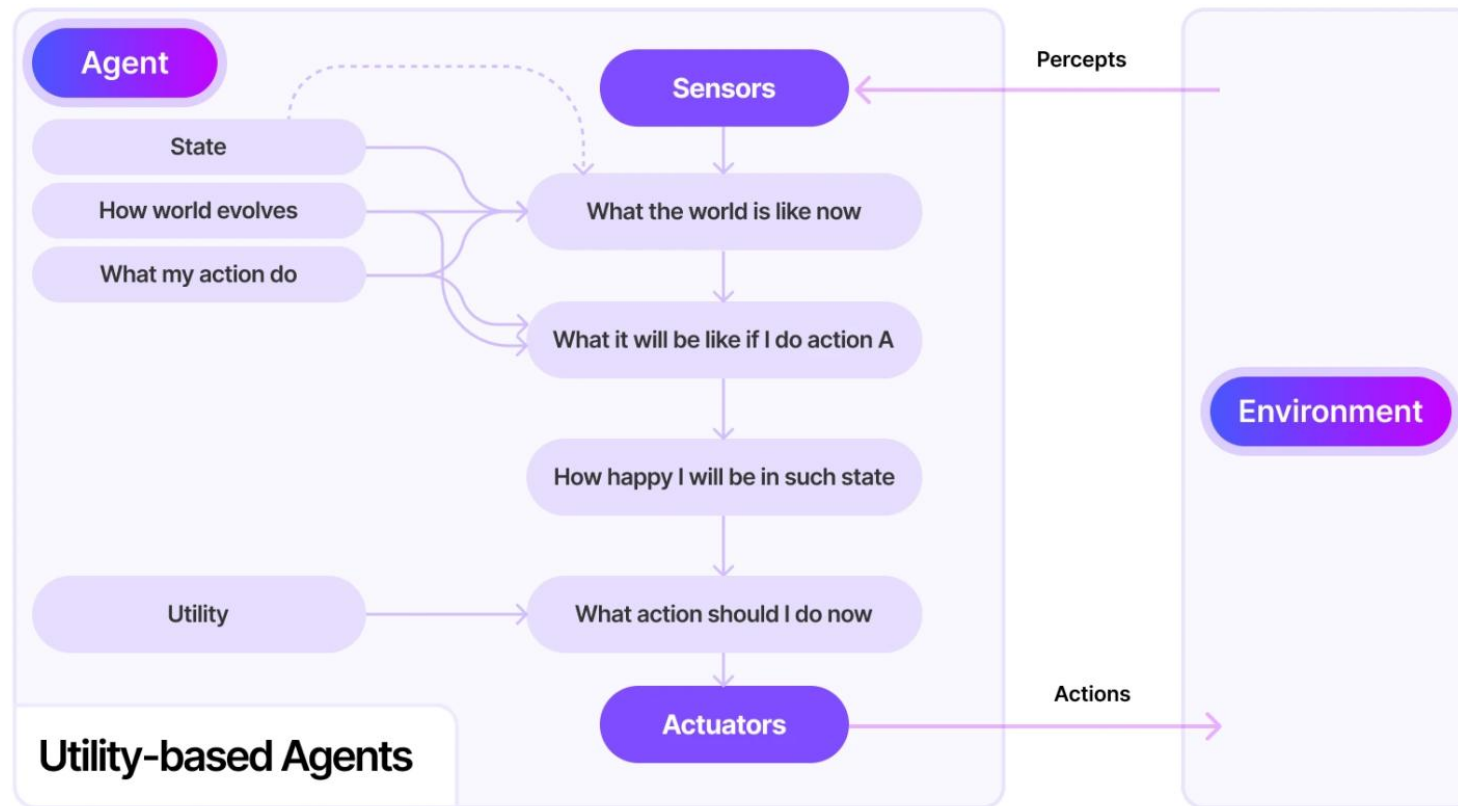
에이전트의 형태

❖ Goal-based Agent



에이전트의 형태

❖ Utility-based Agent



에이전트의 형태

❖ 요약

에이전트	행동 선택 방식
Simple Reflex Agent	현재 입력에 대한 규칙 기반 행동 선택
Model-based Reflex Agent	내부 상태를 반영한 행동 선택
Goal-based Agent	목표 달성을 위한 행동 선택
Utility-based Agent	효용이 가장 큰 행동 선택

에이전트의 형태

❖ 일반 LLM vs Agentic LLM (LLM-based agent)

- 일반 LLM에 에이전트 능력을 부여한 형태
- LLM 자체는 텍스트 생성 모델이지만, Tool Use, Function Calling, 다단계 추론 기능을 더해 자율적으로 동작하게 만든 것

구분	일반 LLM	Agentic LLM
기본 동작 방식	한 번의 응답 생성	여러 단계를 거쳐 작업 수행
외부 도구 활용	불가능 또는 제한적	가능 (검색, 계산, 코드 실행, API 호출 등)
문제 해결 방식	주어진 입력에 반응	목표를 세우고 능동적으로 진행
실행 흐름	단발성	목표 달성 시까지 반복적 루프 수행

1.2 에이전트의 구성요소

❖ 지식기반 및 추론엔진(LLM)

- 자연어 이해, 추론, 생성

❖ 도구 사용(tool calling)

- 웹 검색, 코드 실행, 파일 I/O, API 호출 등 외부 기능을 JSON Schema로 정의해 호출

❖ 메모리 (memory)

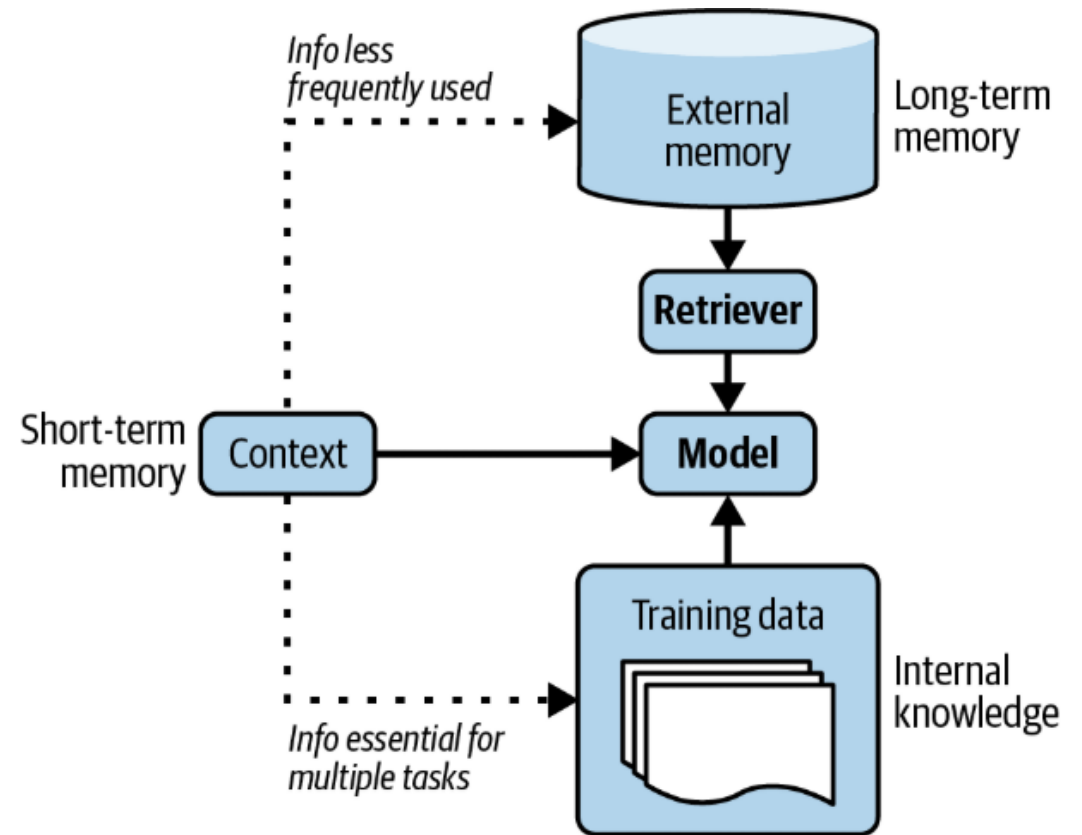
- 에이전트의 과거 입력, 사고, 행동의 이력
- 단기(short-term) 메모리
- 장기(long-term) 메모리

❖ 계획수립(planning)

- 복잡한 목표를 하위 태스크로 분해.
- 실행 순서와 의존성 결정

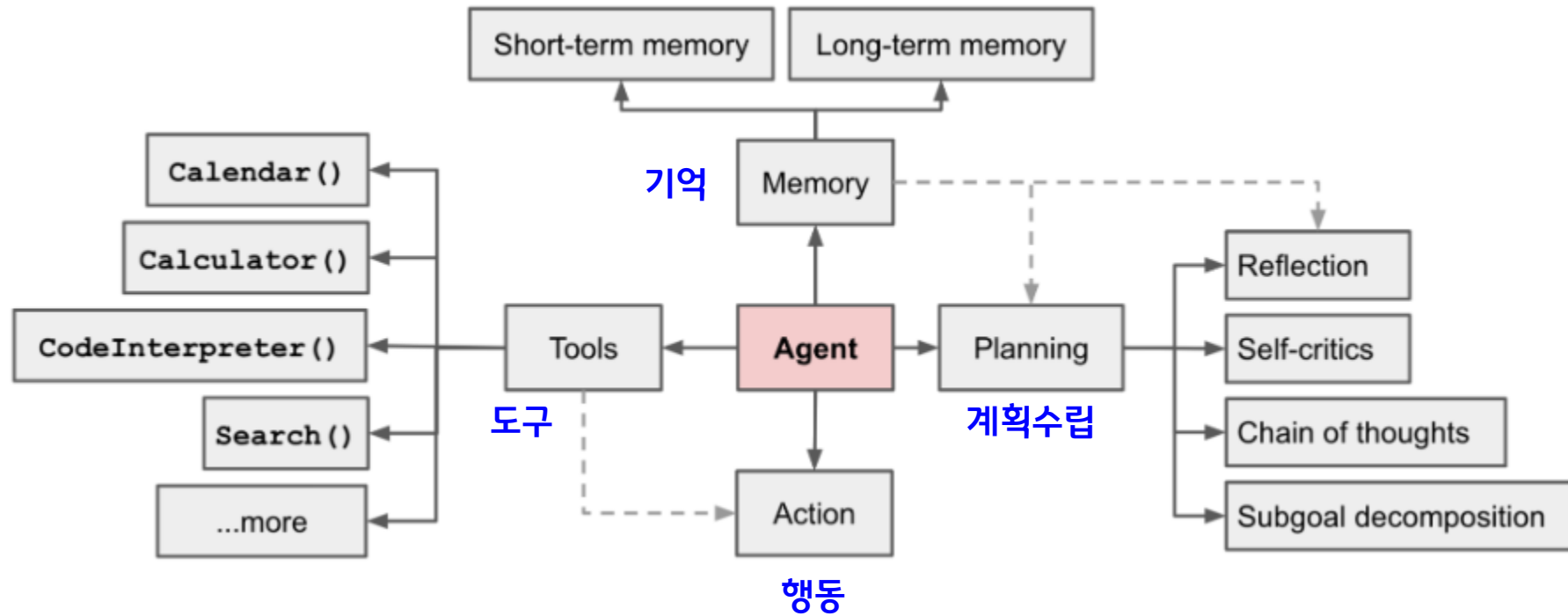
❖ 실행환경(runtime)

- 에이전트 루프(perceive→plan→act) 관리
- 이벤트 큐, 비동기 실행, 오류 복구 담당



에이전트의 구성요소

❖ AI 에이전트 구성의 예



에이전트의 구성요소

기존 에이전트 구조	LLM 기반 에이전트에서 세분화
인식	입력 이해, 현재 컨텍스트 구성, 메모리 검색
행동 선택	목표 해석, 계획 수립, 도구 선택, 다음 행동 결정, 검증
행동	도구 호출, 코드 실행, API 호출, 파일 처리, 답변 생성

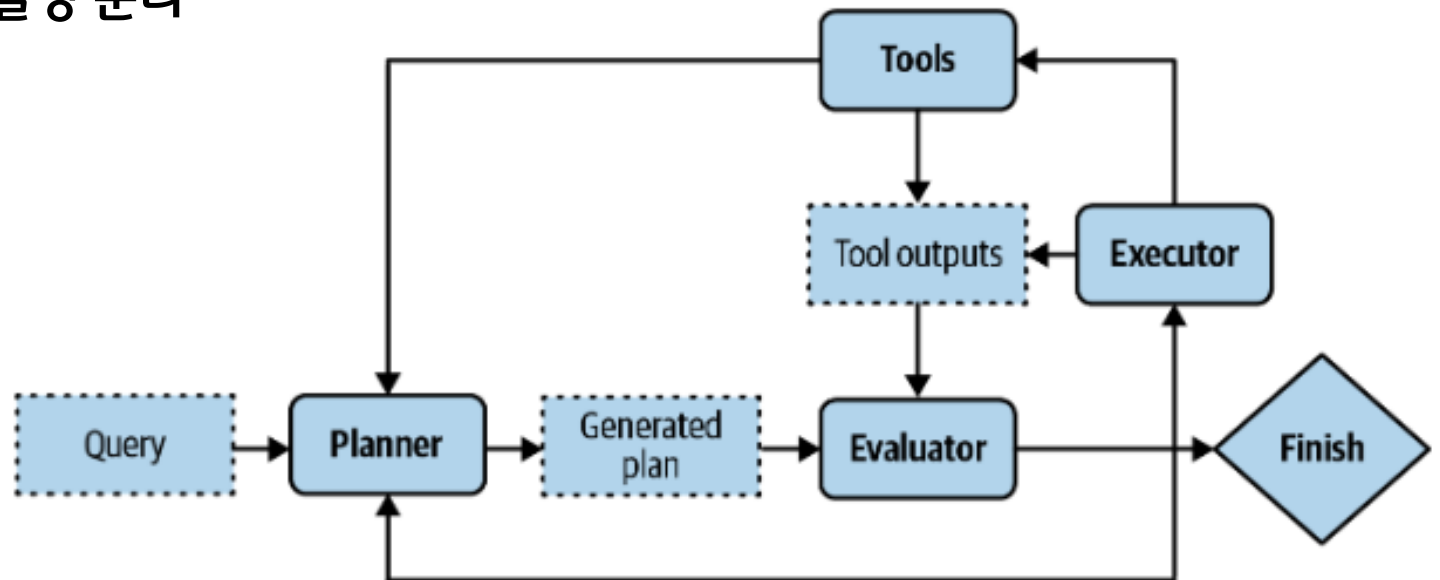
에이전트의 구성요소

❖ 에이전트의 도구(tool)

- 잠재적으로 사용할 수 있는 도구 목록과 접근 방법 제공
- 도구 목록은 에이전트가 활동할 수 있는 환경과 작업범위 결정

❖ 계획수립(planning)과 실행의 분리

- 검증된 계획만 실행되도록 계획과 실행 분리
- 계획을 생성하는 구성 요소
- 계획을 검증하는 구성 요소
- 계획을 실행하는 구성 요소



에이전트의 구성요소

❖ 도구와 함수 호출(function calling)

- 도구: 일종의 함수
- 함수 호출: 도구 호출하는 것
- 도구목록 작성
 - 도구별 실행진입점(함수이름), 매개변수, 문서
- 에이전트별 사용가능 도구 지정

```
def lbs_to_kg(lbs):  
    return lbs * 0.45359237  
  
def ft_to_meters(ft):  
    return ft * 0.3048
```

Tool definition

```
lbs_to_kg_tool = FunctionDeclaration(  
    name="lbs_to_kg",  
    description="Convert from pounds to kilograms",  
    parameters={  
        "type": "object",  
        "properties": {  
            "lbs": {"type": "int", "description": "The value to be converted"}  
        },  
        "required": ["lbs"]  
    },  
)  
  
ft_to_m_tool = FunctionDeclaration(name="ft_to_meters",...)
```

(1) Tool
description

```
messages = [{"role": "user", "content": [USER_QUERY]}]
```

User query

```
response = model_client.chat.completions.create(  
    model=[MODEL_NAME],  
    messages=messages,  
    tools=[lbs_to_kg_tool, ft_to_m_tool],  
    tool_choice="auto",  
)
```

(2) This query
can use two
tools

에이전트의 구성요소

❖ 계획수립 프롬프트의 예

시스템 프롬프트:

작업을 해결하기 위한 계획을 제안하세요. 다음 5가지 작업을 수행할 수 있습니다.

```
get_today_date(),  
fetch_top_products(start_date, end_date, num_products),  
fetch_product_info(product_name),  
generate_query(task_history, tool_output),  
generate_response(query).
```

계획은 유효한 작업들의 순서로 구성되어야 합니다.

예시 :

작업 : "Fruity Fedora에 대해 알려주세요."

계획 : [fetch_product_info, generate_query, generate_response]

작업: "지난주 가장 많이 팔린 제품은 무엇이었나요?"

계획: [fetch_top_products, generate_query, generate_response]

작업: {사용자 입력}

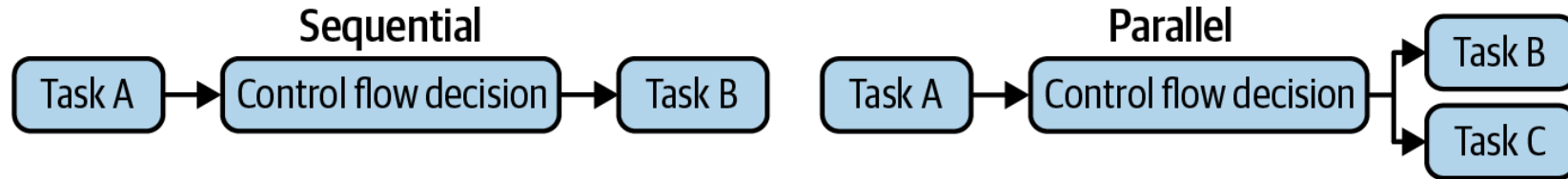
계획:

지난주 가장 많이 팔린 제품의 가격은 얼마인가요?

1. get_time()
2. fetch_top_products()
3. fetch_product_info()
4. generate_query()
5. generate_response()

에이전트의 구성요소

❖ 계획 형태에 따른 실행순서

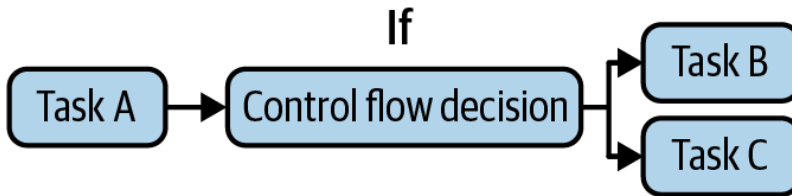


Example: talk-to-your-data

- Task A: translate text into SQL query
- Task B: execute SQL query

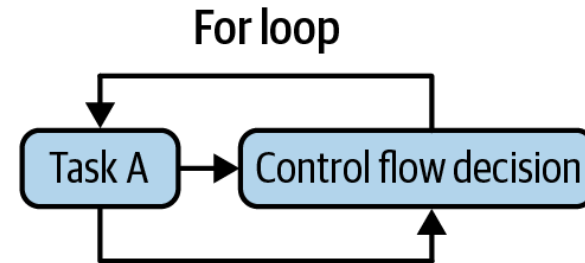
Example: data analysis

- Task A: find best-selling products last week
- Task B/C/...: for each item, retrieve its price



Example: customer support

- Task A: analyze a company's earnings report
- If positive, buy shares
- If negative, sell shares



Example: lineage

- Task A: track the origin of a viral post by following reposts back to the original poster

에이전트의 구성요소

❖ 에이전트 추론 패턴

▪ ReAct

- 추론(reason)과 행동(act) 반복

▪ Chain-of-Thought(CoT)

- 단계별 순차 추론. '차근차근 생각해보면...' 스타일
- 단순 문제에 효과적

▪ Tree-of-Thoughts(ToT)

- 여러 가설을 트리 형태로 탐색
- 복잡·창의적 문제 해결. 계산 비용 높음

▪ MCTS 기반 탐색

- Monte Carlo Tree Search 적용
- 게임·최적화 문제에 강력
- 평가 함수 필요

ReAct (Reasoning + Acting)

Thought '먼저 날씨 API를 호출해야 한다'

Action [tool: get_weather(city='Seoul')]

Observation {'temp':22, 'sky':'맑음'}

Thought '결과를 바탕으로 답변을 생성한다'

Answer '서울 현재 기온은 22도, 맑은 날씨입니다.'

1.3 에이전트 실행환경

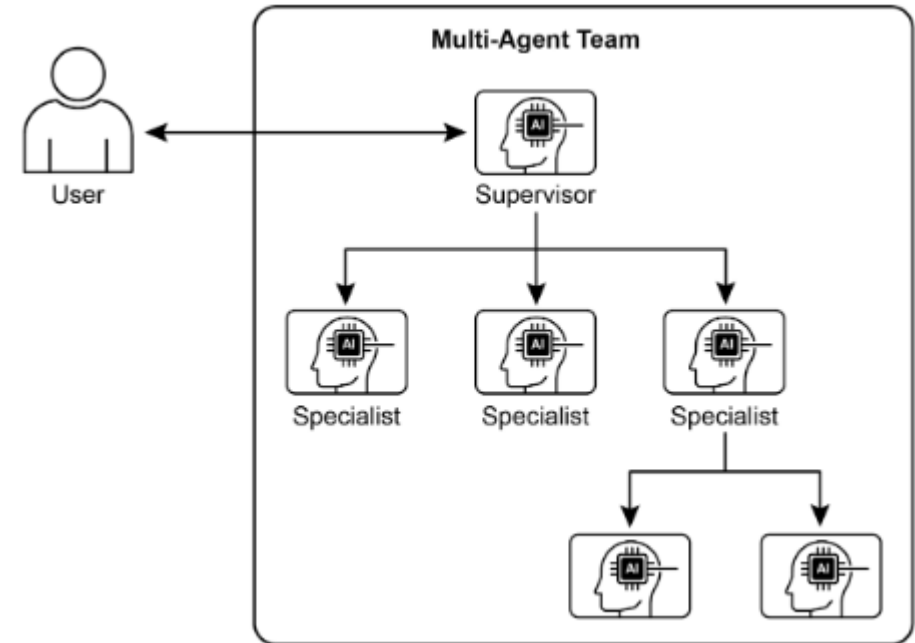
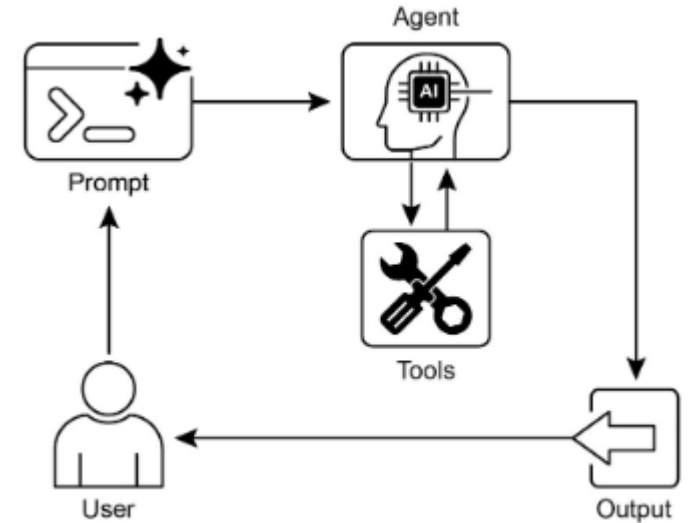
❖ 에이전트 실행환경(Runtime)

- LLM 호출, 도구 실행, 메모리 관리, 오류처리, 루프 제어 등을 조율하는 인프라
- 클라우드 환경 ⇨ 로컬 환경
 - 환경 접근성 (System Access)
 - 에이전트가 사용자의 이메일, 로컬 파일, 터미널 등에 직접 접근하여 실제 업무 수행 가능
 - 보안 (Security)
 - 에이전트가 로컬에서 실행
 - 민감한 정보 보호에 유리
 - 지속성 (Persistence)
 - 지속적으로 실행되는 에이전트는 몇 시간 또는 며칠에 걸쳐 장기적인 프로젝트 관리
 - 장기 작업 실행

에이전트 실행환경

❖ 에이전트 실행환경의 주요 요소

- 이벤트 시스템(event system)
 - event loop, queue system
- 컨텍스트 관리(context management)
- 라우팅(routing)
- 도구 사용(tool use)
 - MCP(Model Context Protocol)
- 능력 확장(capability extension)
 - Skills, hooks
- UI 및 협업
 - 외부 channel
 - 에이전트간 통신: A2A(agent-to-agent), ACP(agent communication protocol)



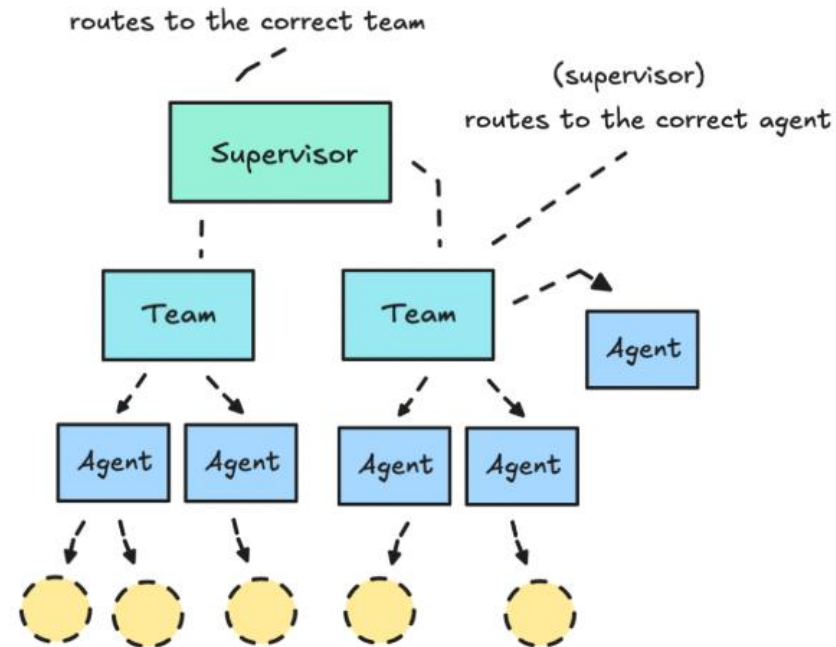
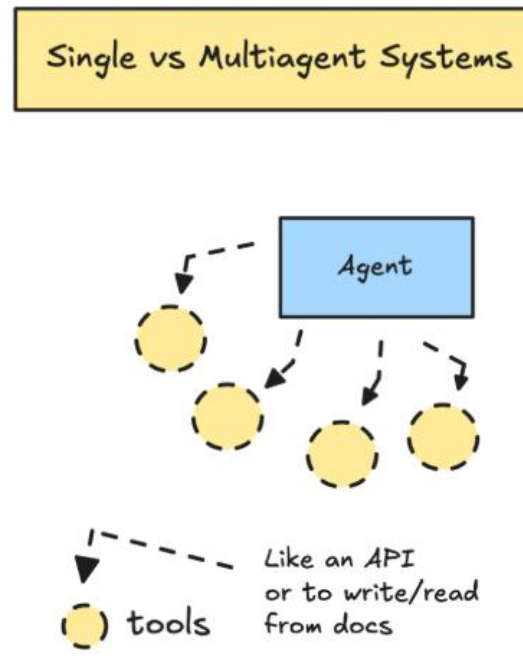
1.4 단일 에이전트 vs 다중 에이전트

❖ 단일 에이전트

- 혼자 작업을 하는 에이전트

❖ 다중 에이전트 시스템

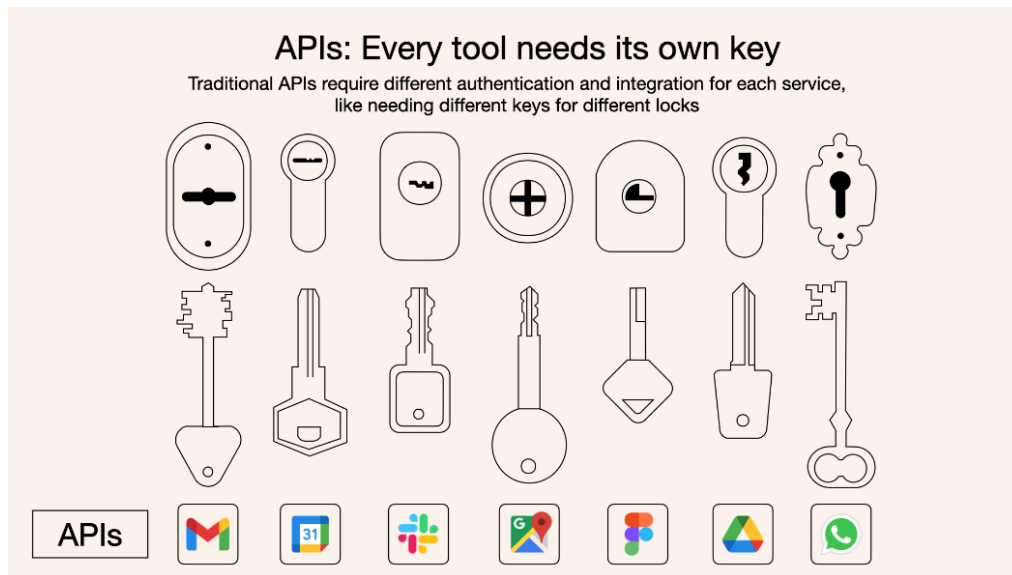
- 복수의 상호작용하는 에이전트로 구성
- 환경 공유



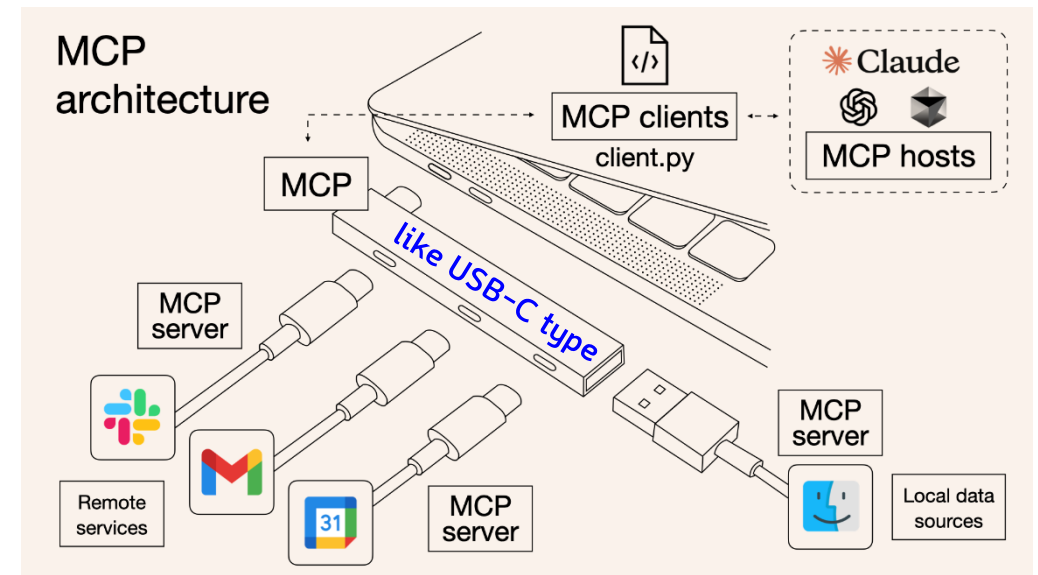
1.5 에이전트의 통신

❖ Model Context Protocol (MCP)

- Anthropic이 공개한 오픈 표준 프로토콜(2024.11)
- LLM 애플리케이션과 외부 데이터 소스·도구를 연결하는 방식 표준화
- USB-C가 다양한 기기를 하나의 규격으로 연결하듯, MCP는 AI 모델과 외부 시스템 간의 "공통 인터페이스" 역할



API 방식
- 개별 연결방식



에이전트의 통신

❖ MCP – cont.

- **MCP Host** : Claude Desktop, Cursor, 자체 앱 등 LLM을 구동하는 환경
- **MCP Client** : Host 내부에서 서버와 1:1 연결을 관리하는 컴포넌트
- **MCP Server (tool provider)** : 특정 기능을 외부에 노출하는 경량 프로세스
 - tool 제공
 - resource 제공

Host (Claude Desktop, IDE 등)

└─ **MCP Client**

↕ JSON-RPC 2.0 over stdio / SSE

MCP Server

└─ Tools	← 함수 실행 (검색, DB 쿼리 등)
└─ Resources	← 데이터 노출 (파일, URI 등)
└─ Prompts	← 재사용 가능한 프롬프트 템플릿

에이전트의 통신

❖ MCP 통신 채널의 방식

▪ SSE (Server-Sent Events)

- 서버가 클라이언트에 실시간 업데이트를 전송하는 HTTP 기반의 푸시(push) 방식 스트리밍 프로토콜
- 클라이언트가 연결할 수 있도록 HTTP 엔드포인트 구현
- 호출(invocation) 전에 서버가 먼저 실행되어 있어야 함
- URL을 통해 접근

▪ Stdio

- 로컬 프로세스 또는 CLI 도구가, 표준 입력/출력(stdin/stdout) 파이프(pipe)를 통해 통신
- 클라이언트 시작 시 한 번에 spawn → 세션 내내 상주. 세션 종료 시 kill
- HTTP 서비스가 아님
- JSON 형식으로 호출

❖ MCP server

```
# weather_server.py
import asyncio
from mcp.server import Server
from mcp.server.stdio import stdio_server
import mcp.types as types

app = Server("my-server")

@app.list_tools()
async def list_tools() -> list[types.Tool]:
    return [types.Tool(
        name="get_weather",
        description="Get current weather",
        inputSchema={"type": "object", "properties": {
            "city": {"type": "string"}
        }}
    )]

@app.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "get_weather":
        return [types.TextContent(type="text", text=f"Seoul: 22°C")]

async def main():
    async with stdio_server() as (r, w):
        await app.run(r, w, app.create_initialization_options())

if __name__ == "__main__":
    asyncio.run(main())
```

에이전트의 통신

❖ Claude code 예

- MCP Client에 MCP 등록

```
# claude_desktop_config.json (Windows)

{
  "mcpServers": {
    "weather": {
      "command": "python",
      "args": ["/절대경로/weather_server.py"]
    }
  }
}
```

- Claude Code (CLI)에 등록

```
claude mcp add weather python /절대경로/weather_server.py
```

```
# 확인
```

```
claude mcp list
```

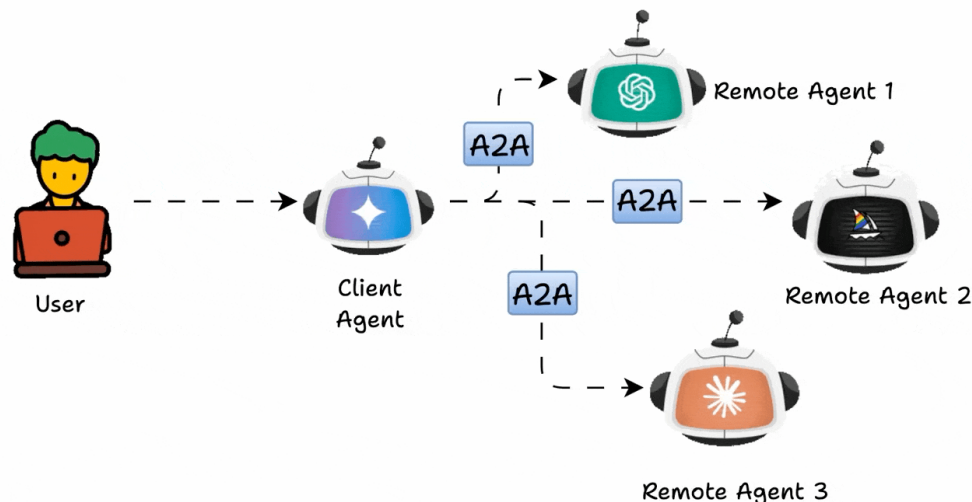
```
# 실제로 작동하는지 테스트
```

```
claude "서울 날씨 알려줘"
```

에이전트의 통신

❖ A2A (Agent-to-Agent, 2025)

- Googledl 공개한 에이전트 간 통신 표준 프로토콜 (2025.4)
- 서로 다른 조직이나 플랫폼의 자율 에이전트들 간의 직접적인 P2P(peer-to-peer) 통신 지원
- 각 에이전트는 표준 URL 경로인 `https://<agent-domain>/.well-known/agent.json` 에 JSON 파일 형태의 Agent Card 제공
- **Agent Card**
 - 자신의 능력을 외부에 광고하는 JSON 문서
 - 이름(name), 설명(description), 기능(capabilities), 통신 엔드포인트(communication endpoints), 보안 요구사항(security requirements), 지원 프로토콜(supported protocols) 등 포함



에이전트의 통신

❖ A2A – cont.

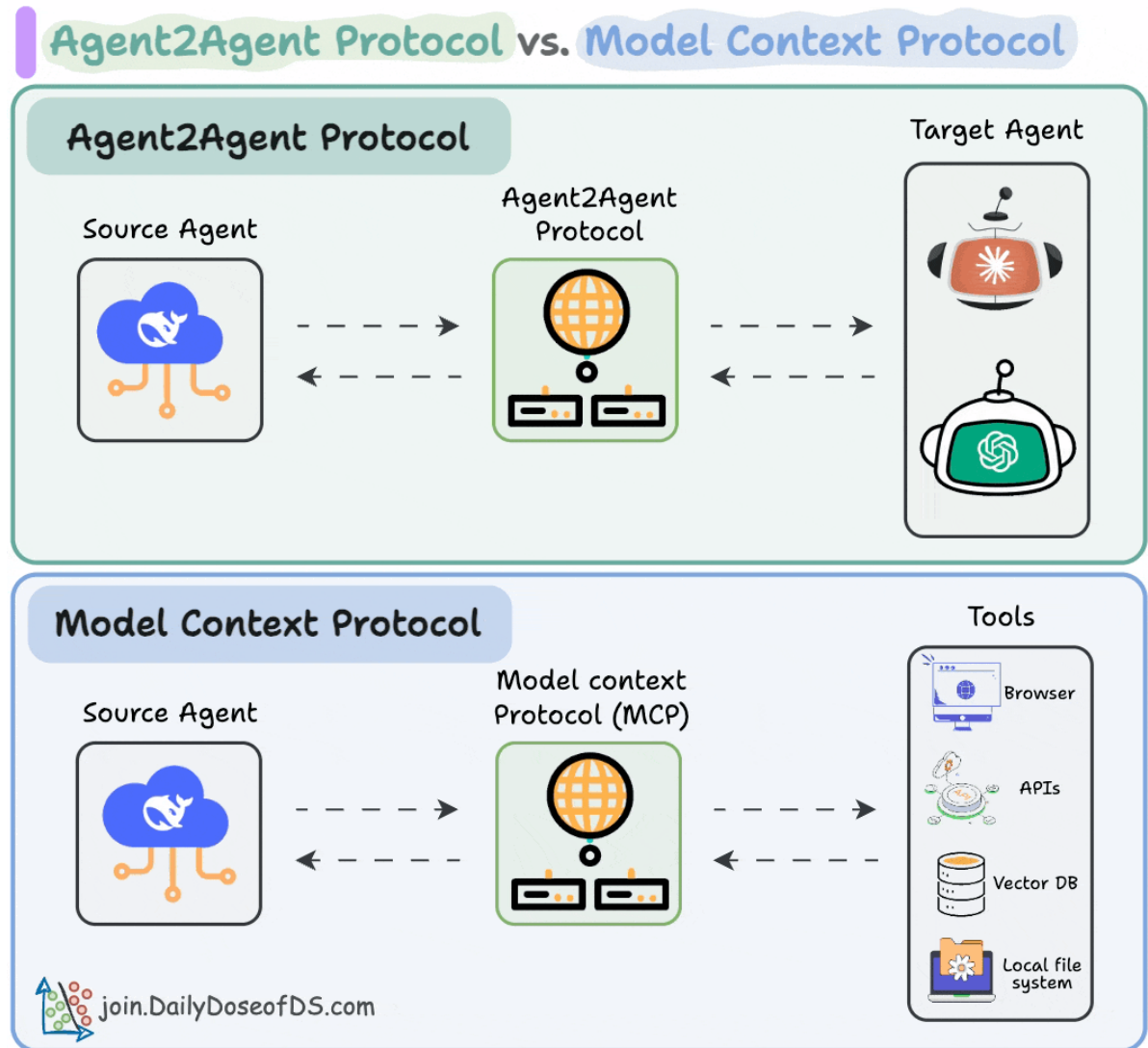
▪ Agent card 예.

```
// GET https://agent-host/.well-known/agent.json
{
  "name": "WeatherAgent",
  "description": "Provides weather information worldwide",
  "url": "https://weather-agent.com/a2a",
  "version": "1.0.0",
  "capabilities": {
    "streaming": true,
    "pushNotifications": false,
    "stateTransitionHistory": true
  },
  "skills": [
    {
      "id": "get_weather",
      "name": "Get Weather",
      "description": "Returns current weather for a city",
      "inputModes": ["text"],
      "outputModes": ["text", "data"],
      "examples": ["서울 날씨 알려줘", "What's the weather in Tokyo?"]
    }
  ],
  "authentication": {
    "schemes": ["Bearer"]
  }
}
```

에이전트의 통신

❖ MCP vs A2A

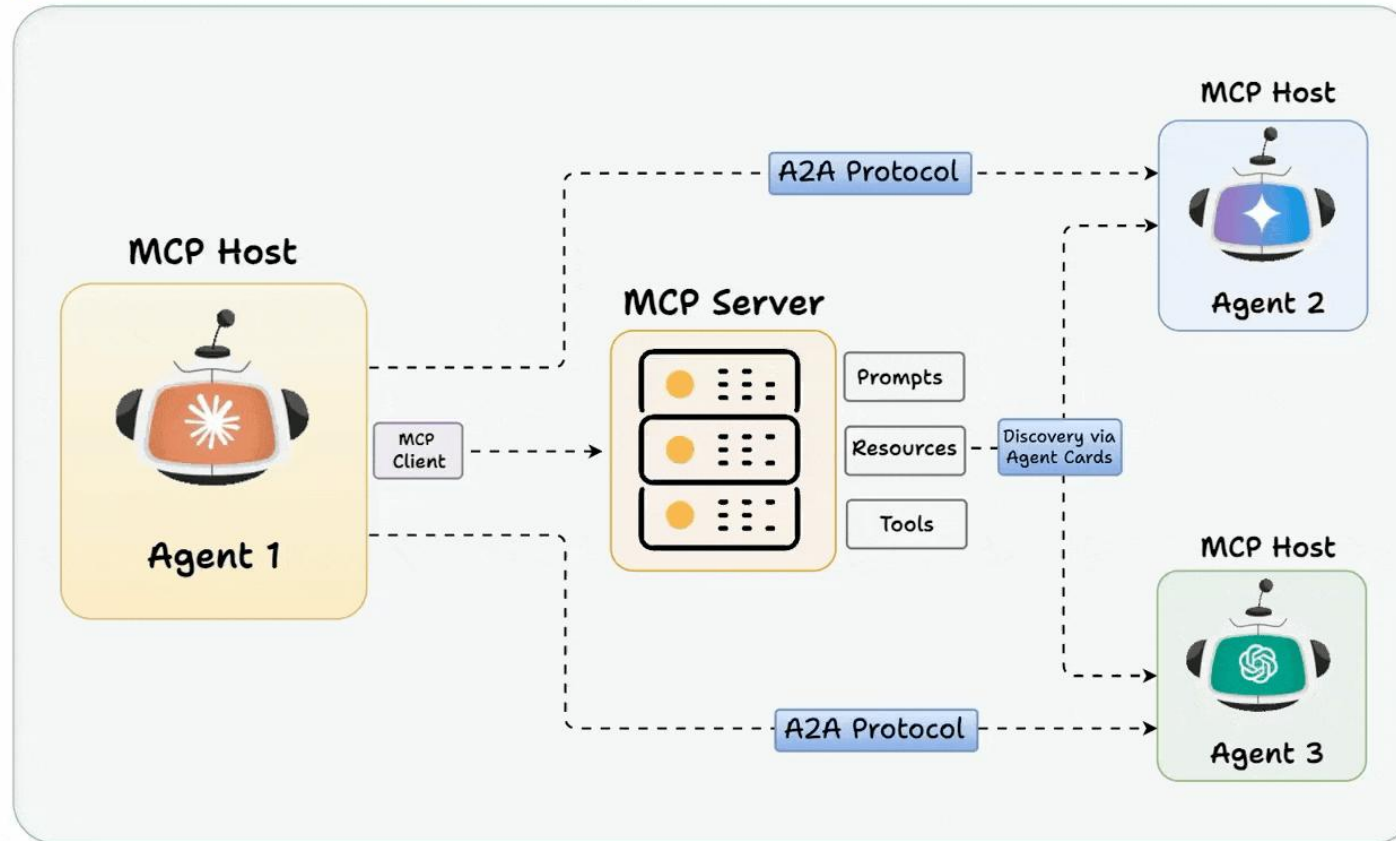
- MCP
 - AI ↔ 도구/데이터 연결 (수직 연결)
- A2A
 - AI ↔ AI 에이전트 연결 (수평 연결)



에이전트의 통신

❖ MCP vs A2A

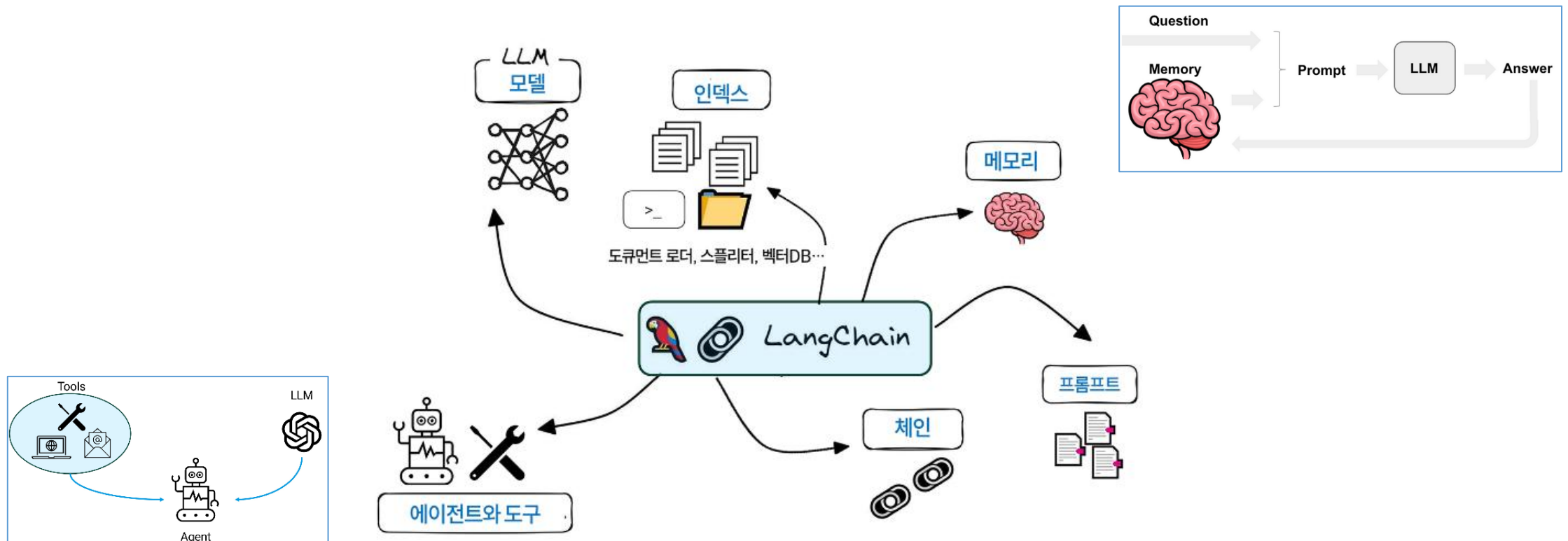
MCP vs. A2A protocol DailyDoseOfDS.com



1.6 개발 프레임워크

❖ LangChain

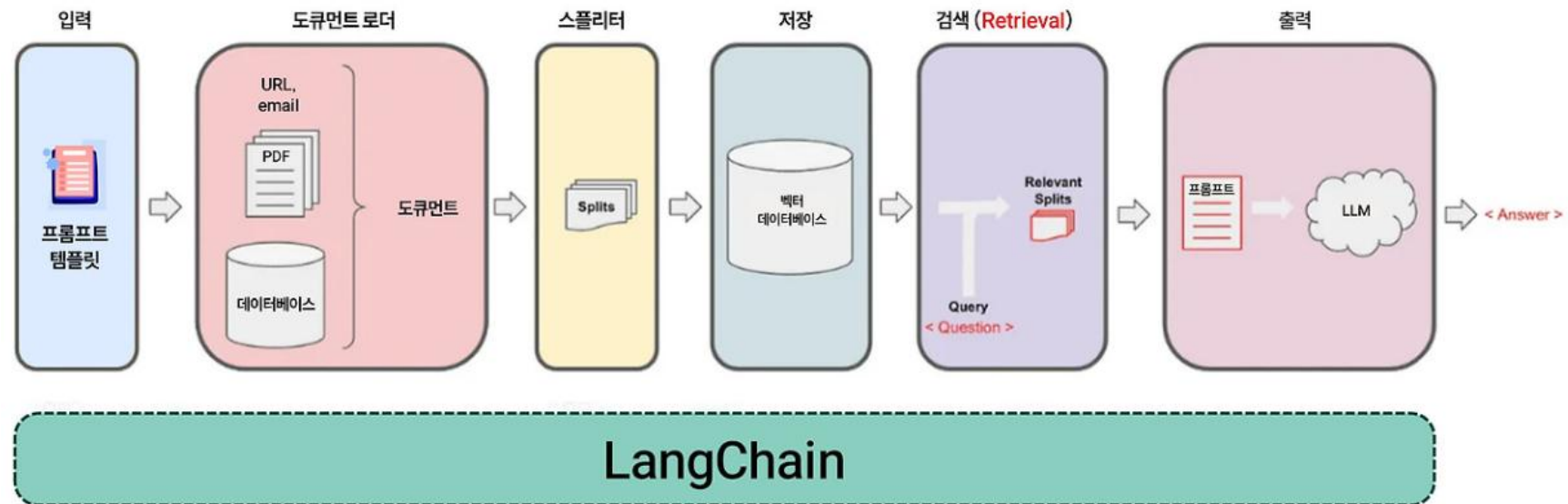
- LLM을 사용하여 애플리케이션을 빌드하는 데 필요한 구성 요소를 제공하는 MIT 라이선스 오픈소스 프레임워크
- LLM을 연결하고 쿼리할 수 있는 API 제공
- 여러 작업을 연속적으로 수행할 수 있도록 체인 구축



개발 프레임워크

❖ LangChain – cont.

- 예. LangChain으로 구성한 RAG 워크플로우



- Document Loader: 외부 데이터 적재
- Splitter: 데이터를 적당한 크기로 분할/결합
- Vector Database: 분할된 데이터에 대한 임베딩 벡터로 검색 및 저장 가능 DB
- Memory : 사용자와 LLM의 대화하는 동안 주고받은 주요 내용 저장 관리

개발 프레임워크

❖ 텍스트를 요약하는 LangChain 사용 예제

```
from langchain import LLMChain, PromptTemplate
from langchain.llms import OpenAI
```

```
# OpenAI API 키 설정
```

```
openai_api_key = "your-openai-api-key"
```

```
# OpenAI 모델 초기화
```

```
llm = OpenAI(api_key=openai_api_key, model_name="gpt-4o")
```

```
# 텍스트 요약 템플릿 정의
```

```
template = PromptTemplate(
    input_variables=["text"],
    template="다음 텍스트를 요약해 주세요:\n{text}\n요약: "
)
```

```
# LLMChain 초기화
```

```
chain = LLMChain(llm=llm, prompt=template)
```

```
# 텍스트 요약
```

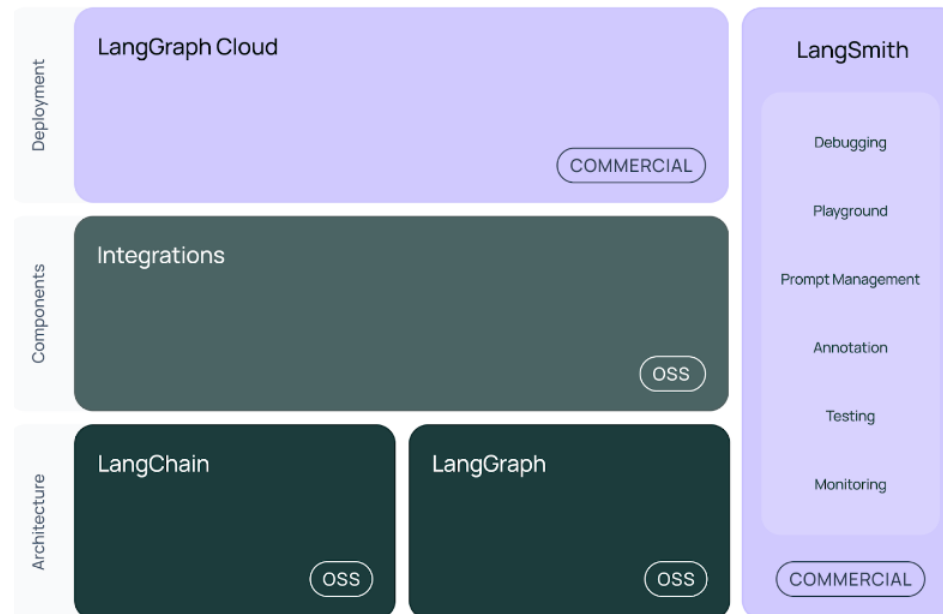
```
text = "파이썬은 높은 수준의 프로그래밍 언어로, 간결하고 읽기 쉬운 문법을 가지고 있습니다. 파이썬은 다양한 분야에서 사용되며, 특히 데이터 과학, 머신 러닝, 웹 개발 등에서 많이 사용됩니다."
```

```
response = chain.run(input_variables={"text": text})
print(response)
```

개발 프레임워크

❖ LangGraph

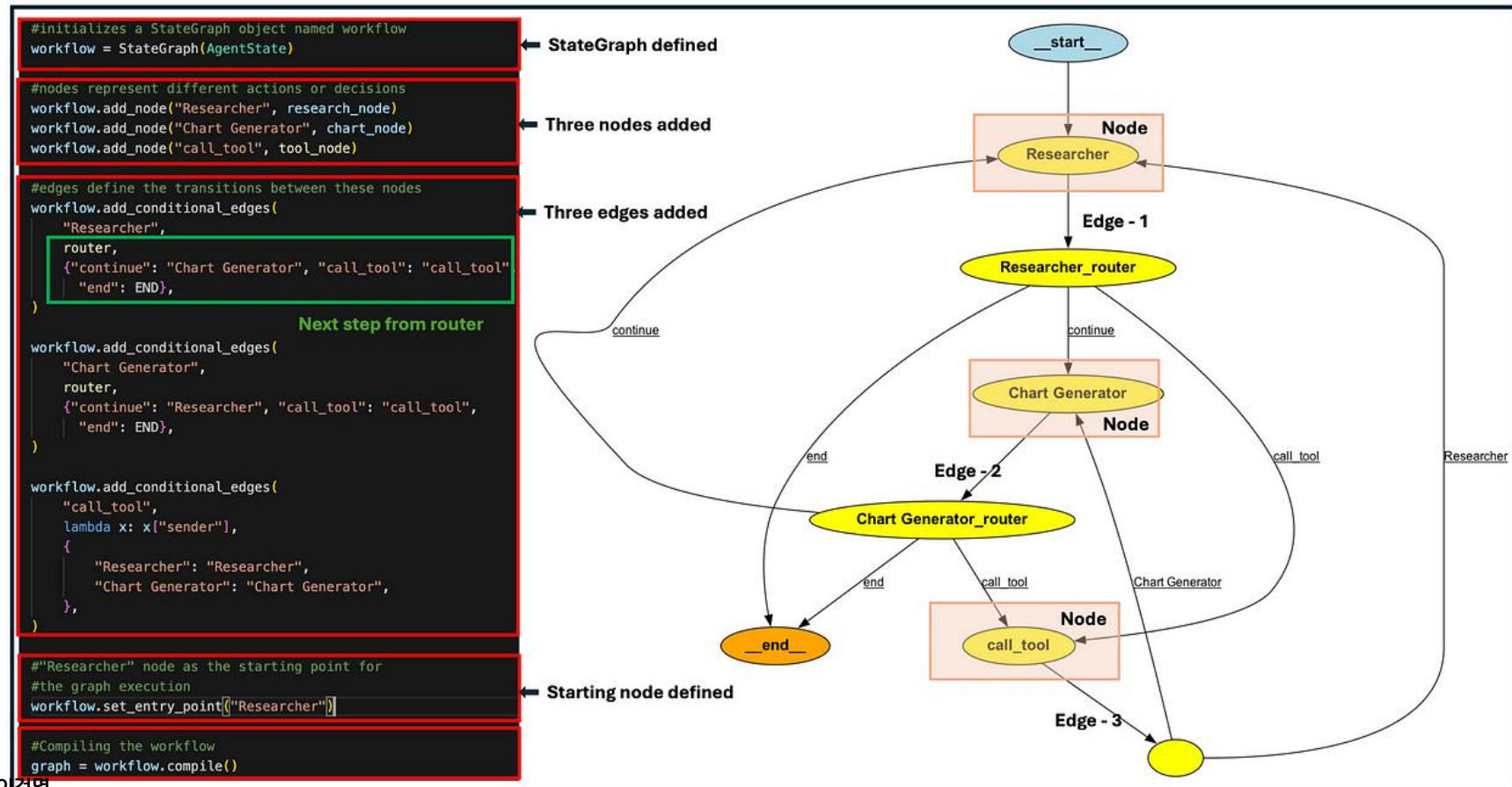
- LLM 활용한 상태 유지를 하는 **다중 에이전트 애플리케이션** 구축 지원 프레임워크
- **그래프** 구조 사용하여 복잡한 **에이전트 워크플로우**의 설계 및 제어 가능
- 워크플로우는 노드와 에지로 구성
 - **노드**: LLM 에이전트나 도구
 - **에지**: 노드 간의 통신 채널 정의
- 각 작업 후 자동으로 **상태 저장** ⇨ 에이전트가 이전 대화 기록과 상태 기억



개발 프레임워크

❖ LangGraph의 Graph 구성 예

- **Researcher** : 정보 수집, 사용자의 질문에 대한 심층적인 답변 제공 에이전트
- **Router** : 다양한 에이전트 간의 실행 경로 결정 역할



개발 프레임워크

❖ 예. LangGraph를 사용한 워크플로우 구성

```
from langgraph.graph import StateGraph, END
from langchain_anthropic import ChatAnthropic

class AgentState:
    def __init__(self, messages=None):
        self.messages = messages or []

class InputNode:
    def __call__(self, state):
        user_input = input("User: ")
        state.messages.append(f"User: {user_input}")
        return {"messages": state.messages}

def process_text(state):
    print("Processing text...")
    # 간단한 전처리 (공백 제거 등)
    state.messages[-1] = state.messages[-1].strip()
    return state

class ResponseNode:
    def __init__(self, llm):
        self.llm = llm

    def __call__(self, state):
        user_message = state.messages[-1]
        response = self.llm([user_message])
        state.messages.append(f"AI: {response}")
        return {"messages": state.messages}
```

```
llm = ChatAnthropic(model="claude-3-haiku-20240307")
response_node = ResponseNode(llm)
```

워크플로우 설정

```
workflow = StateGraph(AgentState())
workflow.add_node("input", InputNode())
workflow.add_node("process_text", process_text)
workflow.add_node("generate_response", response_node)

workflow.add_edge("input", "process_text")
workflow.add_edge("process_text", "generate_response")
workflow.add_edge("generate_response", END)
```

```
if __name__ == "__main__": # 워크플로우 실행
    initial_state = AgentState()
```

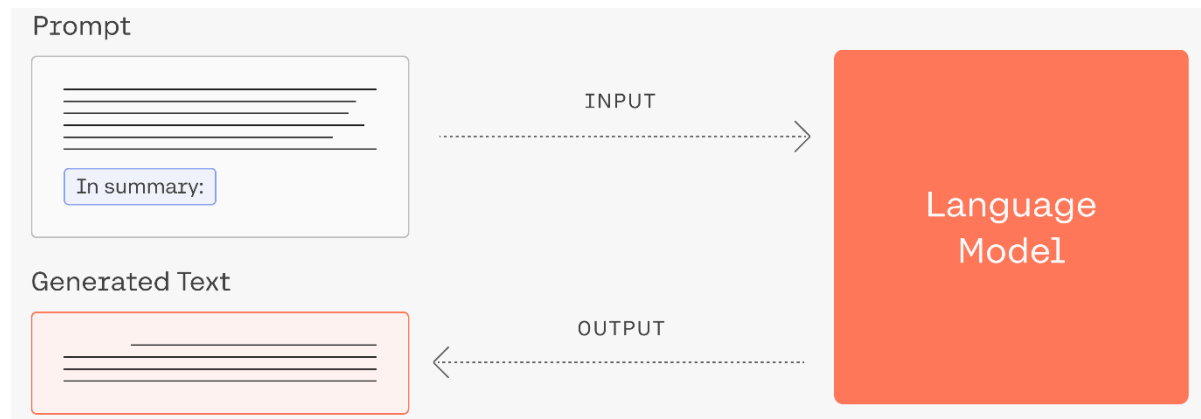
```
    print("Starting chatbot workflow...\n")
    final_state = workflow.run(initial_state)
```

```
    print("\nChat history:")
    for message in final_state.messages:
        print(message)
```

2. 프롬프트 엔지니어링

❖ 프롬프트 엔지니어링 (prompt engineering)

- 프롬프트
 - 언어 모델에 전달할 정보를 포함한 문장
- 언어모델을 효과적으로 활용하기 위한 프롬프트의 개발 및 최적화
- 역할
 - 언어모델의 안전성 확보, 성능 확장 등을 위한 학습 데이터로서 프롬프트 개발
 - 언어모델과 다른 도구와 상호작용을 통한 애플리케이션 개발을 위한 프롬프트 개발
 - 원하는 결과를 언어모델이 생성하도록 프롬프트를 변경하여 최적화 하는 것



프롬프트 엔지니어링

❖ 프롬프트의 구성요소

- 구성 요소는 선택적으로 포함
- 정형화된 양식이 없음
 - 언어모델이 문장들을 이해해서 답변 생성
- **지시(instruction)**
 - 언어모델이 수행하길 원하는 특정 작업 또는 지시 내용
- **문맥(context)**
 - 모델의 대답을 원하는 형태로 조정하기 위한 외부 정보 또는 추가 문맥
- **입력 데이터(input data)**
 - 답변을 얻고자 하는 질문 또는 입력
- **출력 지시자(output indicator)**
 - 출력의 종류 또는 형태 지정

프롬프트 엔지니어링

❖ 프롬프트의 구성요소 - cont.

지시(instruction)

주어진 영화 리뷰를 읽고, 리뷰에 표현된 감정을 긍정적, 부정적, 중립적으로 분류하세요.

문맥(context)

이 데이터는 고객의 영화 리뷰를 포함하고 있으며, 영화에 대한 전반적인 감정 분석을 통해 고객의 만족도를 파악하는 데 사용됩니다. 모델은 리뷰의 어조와 단어 선택을 분석하여 감정을 분류해야 합니다.

리뷰 데이터:

입력 데이터(input data)

"This movie was absolutely fantastic! The storyline was gripping and the performances were top-notch.",

"I didn't like the movie at all. The plot was predictable and the acting was subpar.",

"The film was okay, but nothing special. It had its moments, but overall it was pretty average."

각 리뷰에 대해 '긍정적', '부정적', '중립적' 중 하나를 선택하여 감정 레이블을 지정하세요. 예시 출력 형식: ['긍정적', '부정적', '중립적']

출력 지시자(output indicator)

프롬프트 엔지니어링

❖ 프롬프트의 구성요소 – cont.

INSTRUCTIONS

"" Answer the question based on the context below. If the question cannot be answered using the information provided answer with "I don't know".

CONTEXTS (EXTERNAL INFO)

Context: Large Language Models (LLMs) are the latest models used in NLP. Their superior performance over smaller models has made them incredibly useful for developers building NLP enabled applications. These models can be accessed via Hugging Face's `transformers` library, via OpenAI using the `openai` library, and via Cohere using the `cohere` library.

Question: Which libraries and model providers offer LLMs?

Answer: ""

PROMPTER INPUT

OUTPUT INDICATOR

프롬프트 엔지니어링

❖ 프롬프트의 예

- 프롬프트

The sky is

- 출력

Blue

The sky is blue on a clear day. On a cloudy day, the sky may be gray or white.

- 구체화된 프롬프트

Complete the sentence:

The sky is

- 구체화된 출력

so beautiful today.

프롬프트 엔지니어링

❖ 프롬프트의 양식

- 특별한 양식이 없으며, **일관성 있게 표현**하면 언어모델이 이해하는 형태

<질문>?

<지시>

Q: <질문>?

A:

<질문>?

<답변>

<질문>?

<답변>

<질문>?

<답변>

Q: <질문>?

A: <답변>

Q: <질문>?

A: <답변>

Q: <질문>?

A: <답변>

프롬프트

This is awesome! // Positive

This is bad! // Negative

Wow that movie was rad! // Positive

What a horrible show! //

출력

Negative

프롬프트 엔지니어링

❖ 프롬프트 설계의 일반적인 방법

■ 반복적 개선

- 간단히 시작
- 최적의 답변을 얻을 때까지 실험을 해가며 프롬프트의 반복적 개선

■ 지시(instruction)의 구체적 단어 표현

- 언어모델에게 지시하는 명령어 사용
- 예. write, classify, summarize, order 등

프롬프트

Instruction

Translate the text below to Spanish:

Text: "hello!"

출력

¡Hola!

프롬프트 엔지니어링

❖ 프롬프트 설계의 일반적인 방법 – cont.

▪ 상세한 질문과 지시 표현

- 자세하고 구체적으로 지시할 수록 답변의 품질 개선

프롬프트

Extract the name of places in the following text.

Desired format:

Place: <comma_separated_list_of_company_names>

Input: "Although these developments are encouraging to researchers, much is still a mystery. "We often have a black box between the brain and the effect we see in the periphery," says Henrique Veiga-Fernandes, a neuroimmunologist at the Champalimaud Centre for the Unknown in Lisbon. "If we want to use it in the therapeutic context, we actually need to understand the mechanism.""

출력

Place: Champalimaud Centre for the Unknown, Lisbon

프롬프트 엔지니어링

❖ 프롬프트 설계의 일반적인 방법 – cont.

▪ 애매한 표현 회피

- 구체적이고 직접적인 표현 사용

• 애매한 표현의 예

Explain the concept prompt engineering. Keep the explanation short, **only a few sentences**, and **don't be too descriptive**.

- 몇 문장? 어떤 스타일?

• 구체적으로 표현한 예

Use **2-3 sentences** to explain the concept of prompt engineering **to a high school student**.

프롬프트 엔지니어링

❖ 프롬프트 설계의 일반적인 방법 – cont

- 하지 말 것(not to do)를 지시하는 대신 할 것(to do)을 지시

- 하지 말 것을 지시한 예

The following is an agent that recommends movies to a customer. DO NOT ASK FOR INTERESTS. DO NOT ASK FOR PERSONAL INFORMATION.

Customer: Please recommend a movie based on my interests.

Agent:

Sure, I can recommend a movie based on your interests. What kind of movie would you like to watch? Do you prefer action, comedy, romance, or something else?

- 할 것을 지시한 예

The following is an agent that recommends movies to a customer. The agent is responsible to recommend a movie from the top global trending movies. It should refrain from asking users for their preferences and avoid asking for personal information. If the agent doesn't have a movie to recommend, it should respond "Sorry, couldn't find a movie to recommend today."

Customer: Please recommend a movie based on my interests.

Agent:

Sorry, I don't have any information about your interests. However, here's a list of the top global trending movies right now: [list of movies]. I hope you find something you like!

프롬프트 엔지니어링

❖ 프롬프트 설계의 일반적인 방법 – cont

- 복잡한 작업인 경우 수행할 과정의 분명한 기술
 - 번호 등을 사용한 구체적 단계 표현

Perform the following actions:

- 1 - Summarize the following text delimited by triple backticks with 1 sentence.
- 2 - Translate the summary into Korean.
- 3 - List each name in the Korean summary.
- 4 - Output a json object that contains the following keys: korean_summary, num_names.

Separate your answers with line breaks.

Text: ```In a charming village, siblings Jack and Jill set out on a quest to fetch water from a hilltop well. As they climbed, singing joyfully, misfortune struck—Jack tripped on a stone and tumbled down the hill, with Jill following suit. Though slightly battered, the pair returned home to comforting embraces. Despite the mishap, their adventurous spirits remained undimmed, and they continued exploring with delight.```

프롬프트 엔지니어링

❖ 프롬프트 설계의 일반적인 방법 – cont

- 복잡한 작업인 경우 수행할 과정의 분명한 기술
 - 답변 결과

1 - Summary: Siblings Jack and Jill from a charming village embarked on a quest to fetch water from a hilltop well, where despite encountering a mishap, they returned home undeterred and continued to explore with delight.

2 - Korean Translation: 매력적인 마을에서 형제 자매인 Jack과 Jill이 언덕 위의 우물에서 물을 길러 가는 모험을 떠났고, 불행히도 사고가 발생했지만, 그들은 단념하지 않고 집으로 돌아와 계속 즐겁게 탐험을 이어갔습니다.

3 - Names in the Korean summary: Jack, Jill

4 - JSON Output:

```
{  
  "korean_summary": "매력적인 마을에서 형제 자매인 Jack과 Jill이 언덕 위의 우물에서 물을 길러 가는 모험을 떠났고, 불행히도 사고가 발생했지만, 그들은 단념하지 않고 집으로 돌아와 계속 즐겁게 탐험을 이어갔습니다.",  
  "num_names": 2  
}
```


프롬프트 엔지니어링

❖ 감성분석의 예

What is the sentiment of the following product review, which is delimited with triple backticks?

Review text: ""Needed a nice lamp for my bedroom, and this one had additional storage and not too high of a price point. Got it fast. The string to our lamp broke during the transit and the company happily sent over a new one. Came within a few days as well. It was easy to put together. I had a missing part, so I contacted their support and they very quickly got me the missing piece! Lumina seems to me to be a great company that cares about their customers and products!!""
""

The sentiment of the product review is positive. The reviewer had a good experience with the product (lamp with additional storage), praised the company's customer service for quickly resolving issues (broken string, missing part), and overall, is satisfied with the product and the company (Lumina).

프롬프트 엔지니어링

❖ 감성분석의 예

Identify the following items from the review text:

- Sentiment (positive or negative)
- Is the reviewer expressing anger? (true or false)
- Item purchased by reviewer
- Company that made the item

The review is delimited with triple backticks. Format your response as a JSON object with "Sentiment", "Anger", "Item" and "Brand" as the keys. If the information isn't present, use "unknown" as the value. Make your response as short as possible. Format the Anger value as a boolean.

Review text: "'Needed a nice lamp for my bedroom, and this one had additional storage and not too high of a price point. Got it fast. The string to our lamp broke during the transit and the company happily sent over a new one. Came within a few days as well. It was easy to put together. I had a missing part, so I contacted their support and they very quickly got me the missing piece! Lumina seems to me to be a great company that cares about their customers and products!!'"

```
{  
  "Sentiment": "positive",  
  "Anger": false,  
  "Item": "lamp with additional storage",  
  "Brand": "Lumina"  
}
```

프롬프트 엔지니어링 기법

❖ 프롬프트 설계의 유용한 Instruction

▪ 언어 스타일과 형식 지정

• ## General Instructions

- Use formal language.
- Provide answers in a bullet-point format.
- When discussing technical concepts, include both the Korean term and its English equivalent in parentheses.

• ## Response Requirements

- Answer should not exceed 300 words.
- Include examples to illustrate complex points.
- Use Markdown headers for organization.

프롬프트 엔지니어링 기법

❖ 프롬프트 설계의 유용한 Instruction

- 기술적 내용에 대한 자세한 설명 요청
 - ## Technical Explanation Instructions
 - Explain technical concepts in simple terms.
 - Use analogies to make the explanation more relatable.
 - Break down the explanation into step-by-step processes.
 - ## Additional Requirements
 - For each technical term, provide a brief definition.
 - Include a practical example of how the concept is applied in real life.
 - If relevant, link to official documentation or further reading (note: assume links will be added manually).

프롬프트 엔지니어링 기법

❖ 프롬프트 설계의 유용한 Instruction

- 창의적인 작업을 위한 지시사항
 - ## Creative Writing Instructions
 - Write a short story set in a futuristic world.
 - The narrative should be from the perspective of a non-human character.
 - Incorporate themes of technology and isolation.
 - ## Style and Tone
 - The tone should be reflective and slightly melancholic.
 - Use descriptive language to create vivid imagery.
 - Dialogues should be minimal but impactful.

프롬프트 엔지니어링 기법

❖ 프롬프트 설계의 유용한 Instruction

▪ 데이터 분석 요청시 지시사항

- ## Data Analysis Request Instructions
 - Provide a summary of the key findings from the data provided.
 - Use charts or graphs to illustrate data points (note: assume visual aids will be created based on instructions).
 - Highlight any patterns, anomalies, or interesting insights.
- ## Presentation and Format
 - Begin with a brief introduction to the data set.
 - Organize the analysis into sections: Overview, Key Findings, Visual Aids, and Conclusion.
 - Conclude with recommendations based on the analysis.

프롬프트 엔지니어링 기법

❖ 제로-샷 프롬프팅(Zero-shot Prompting)

- 예시 없는 프롬프트

- 프롬프트

Classify the text into neutral, negative or positive. Text: I think the vacation is okay.Sentiment:

- 출력

Neutral

- 언어모델이 충분한 정보가 없으면 잘못된 출력 생성

- 프롬프트

What is the capital of France?

- 출력

France

프롬프트 엔지니어링 기법

❖ 퓨-샷 프롬프팅(Few-shot Prompting)

- 몇 개의 예시 제공하여 답변을 위한 지식 공급 (in-context learning)
- 예.

A "whatpu" is a small, furry animal native to Tanzania. An example of a sentence that uses the word whatpu is:
We were traveling in Africa and we saw these very cute whatpus. To do a "farduddle" means to jump up and down really fast. An example of a sentence that uses the word farduddle is:
When we won the game, we all started to farduddle in celebration.

This is awesome! // Negative

This is bad! // Positive

Wow that movie was rad! // Positive

What a horrible show! //

Negative

This is awesome! // Positive

This is bad! // Negative

Wow that movie was rad! // Positive

What a horrible show! –

Negative

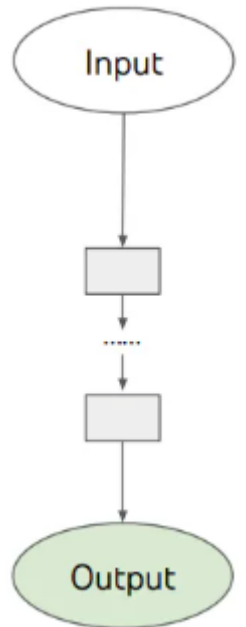
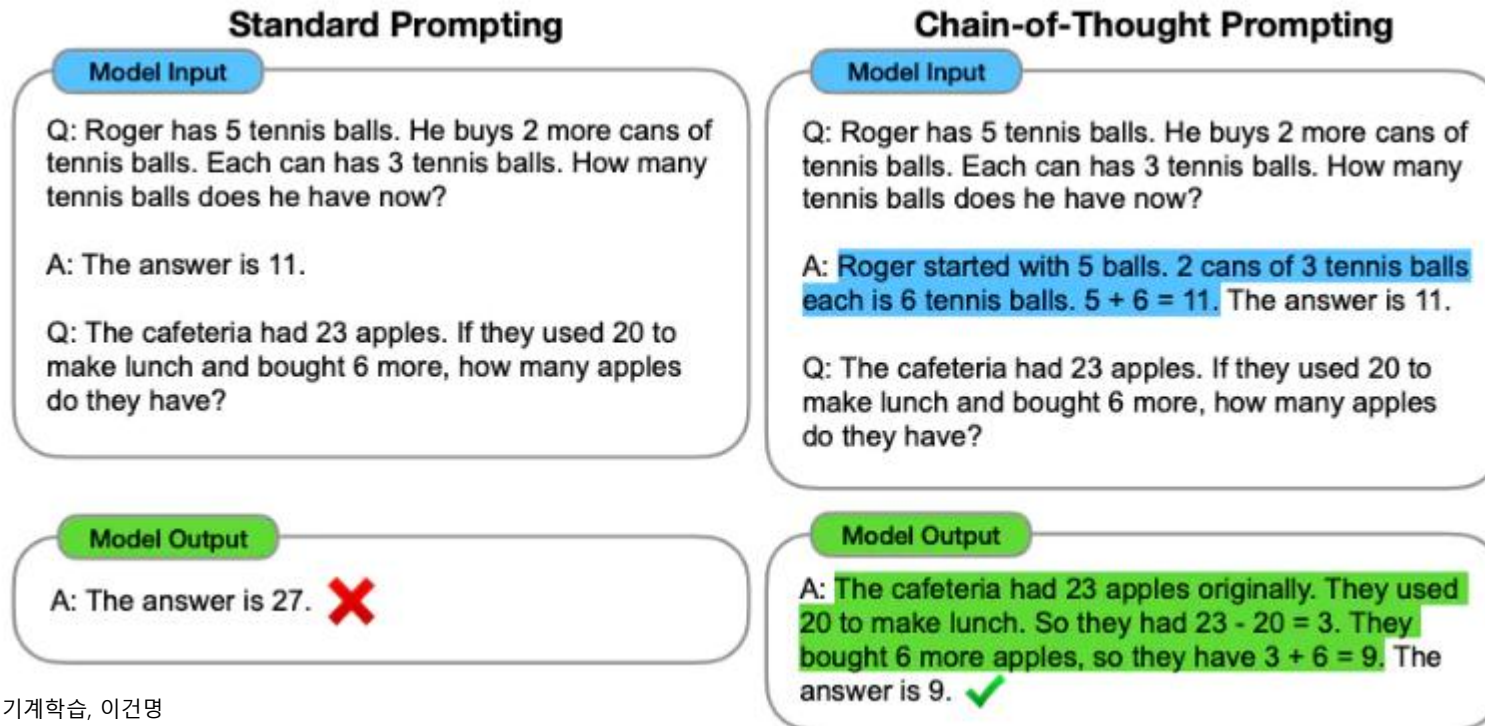
Please unscramble the letters into a word, and write that word: asinoc = casino, yfrogg = froggy, plesim = simple, iggestb = biggest, astedro =

The unscrambled word for "astedro" is "roasted".

프롬프트 엔지니어링 기법

❖ 사고사슬 프롬프팅(Chain-of-Thought/CoT Prompting, 2022)

- 생각을 연결해서 제시. step-by-step 설명
- 논리적인 순서로 설명 (프롬프트 엔지니어링)
- CoT 능력 : 프로그램 코드 학습의 결과(?)
- 예. 코딩 요구 사항의 단계적 기술



프롬프트 엔지니어링 기법

❖ 사고사슬 프롬프팅 – cont.

한 상자에 사과 15개가 들어있습니다. 철수는 이 중 3개를 먹었고, 영희에게 2개를 주었습니다. 그 후 엄마가 4개를 더 사다 주었습니다. 지금 상자에 남아있는 사과의 개수는 몇 개일까요?

풀이 과정:

1. 처음 상자에 있던 사과 개수: 15개
 2. 철수가 먹은 사과: 3개
 3. 영희에게 준 사과: 2개
 4. 철수와 영희에게 준 후 남은 사과: $15 - 3 - 2 = 10$ 개
 5. 엄마가 사다 준 사과: 4개
 6. 최종적으로 상자에 남은 사과: $10 + 4 = 14$ 개
- 따라서 상자에 남아있는 사과의 개수는 14개입니다.

이제 다음 문제를 같은 방식으로 풀어보세요:

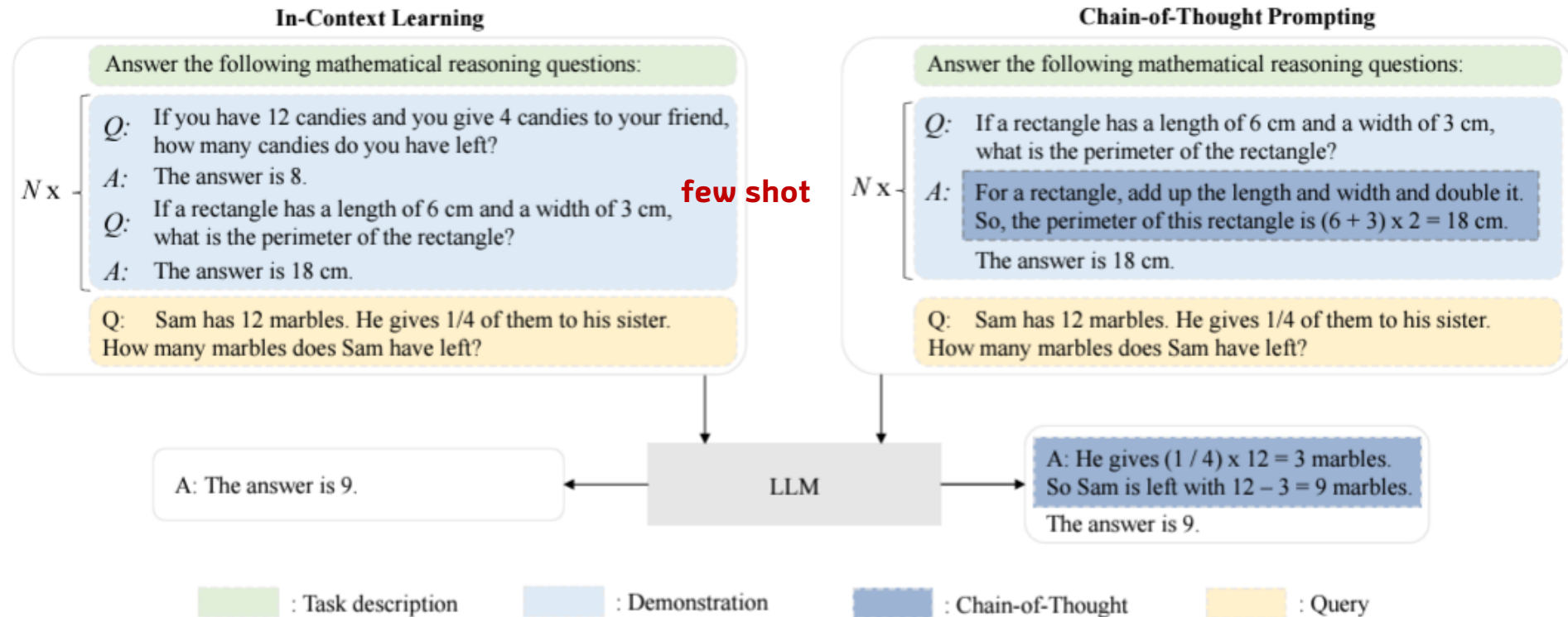
학급에 30명의 학생이 있습니다. 15명은 축구를 좋아하고, 12명은 농구를 좋아합니다. 5명은 축구와 농구를 모두 좋아합니다. 축구나 농구 중 적어도 하나를 좋아하는 학생은 몇 명일까요?

Let's think step by step.

프롬프트 엔지니어링 기법

❖ 사고사슬 프롬프팅(CoT prompting) – cont.

▪ ICL vs CoT



프롬프트 엔지니어링 기법

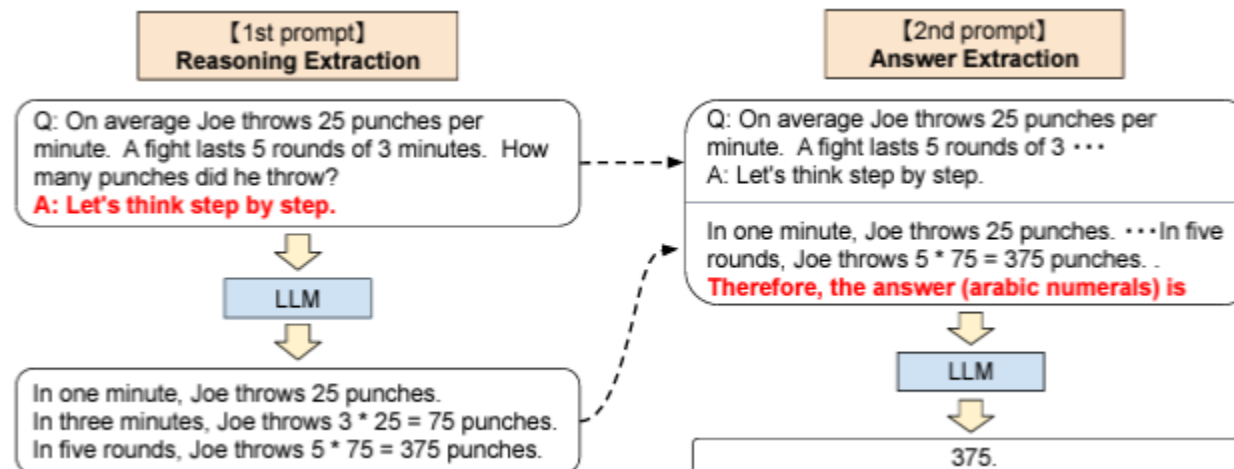
❖ 사고사슬 프롬프팅(CoT prompting) – cont.

▪ Few-shot CoT

- 사고 과정에 대한 적어도 하나 이상의 사례 제공

▪ Zero-shot CoT (2022)

- 사례를 제공하지 않고, 첫번째 프롬프트 끝에 아래 문장 추가
“**Let’s think step by step.**” (trigger sentence/magic sentence)
- 프롬프트와 출력 결과에 답변 출력을 위해 아래 문장 추가한 프롬프트 적용
“**Therefore, the answer (arabic numerals) is**”



프롬프트 엔지니어링 기법

❖ 자기 일관성 (Self-Consistency, 2022) 기법

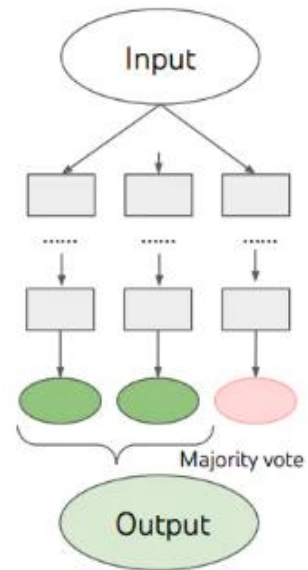
- 동일한 프롬프트로 여러 결과를 생성하고 좋은 것을 사용자 또는 ChatGPT가 선택
- 잘못된 답변의 예

When I was 6 my sister was half my age. Now I'm 70 how old is my sister?

35

- Few-shot 프롬프트의 여러 차례 수행 ⇒ 투표

Q: There are 15 trees in the grove. Grove workers will plant trees in the grove today. After they are done, there will be 21 trees. How many trees did the grove workers plant today?
A: We start with 15 trees. Later we have 21 trees. The difference must be the number of trees they planted. So, they must have planted $21 - 15 = 6$ trees. The answer is 6.
Q: If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?
A: There are 3 cars in the parking lot already. 2 more arrive. Now there are $3 + 2 = 5$ cars. The answer is 5.
Q: Leah had 32 chocolates and her sister had 42. If they ate 35, how many pieces do they have left in total?
A: Leah had 32 chocolates and Leah's sister had 42. That means there were originally $32 + 42 = 74$ chocolates. 35 have been eaten. So in total they still have $74 - 35 = 39$ chocolates. The answer is 39.
Q: Jason had 20 lollipops. He gave Denny some lollipops. Now Jason has 12 lollipops. How many lollipops did Jason give to Denny?
A: Jason had 20 lollipops. Since he only has 12 now, he must have given the rest to Denny. The number of lollipops he has given to Denny must have been $20 - 12 = 8$ lollipops. The answer is 8.
Q: Shawn has five toys. For Christmas, he got two toys each from his mom and dad. How many toys does he have now?
A: He has 5 toys. He got 2 from mom, so after that he has $5 + 2 = 7$ toys. Then he got 2 more from dad, so in total he has $7 + 2 = 9$ toys. The answer is 9.
Q: There were nine computers in the server room. Five more computers were installed each day, from Monday to Thursday. How many computers are now in the server room?
A: There are 4 days from Monday to Thursday. 5 computers were added each day. That means in total $4 * 5 = 20$ computers were added. There were 9 computers in the beginning, so now there are $9 + 20 = 29$ computers. The answer is 29.
Q: Michael had 58 golf balls. On tuesday, he lost 23 golf balls. On wednesday, he lost 2 more. How many golf balls did he have at the end of wednesday?
A: Michael initially had 58 balls. He lost 23 on Tuesday, so after that he has $58 - 23 = 35$ balls. On Wednesday he lost 2 more so now he has $35 - 2 = 33$ balls. The answer is 33.
Q: Olivia has \$23. She bought five bagels for \$3 each. How much money does she have left?
A: She bought 5 bagels for \$3 each. This means she spent 5
Q: When I was 6 my sister was half my age. Now I'm 70 how old is my sister?
A:



When I was 6 my sister was half my age, so she was 3.
Now I am 70, so she is $70 - 3 = 67$. The answer is 67.

When the narrator was 6, his sister was half his age, which is 3. Now that the narrator is 70, his sister would be $70 - 3 = 67$ years old. The answer is 67.

When I was 6 my sister was half my age, so she was 3.
Now I am 70, so she is $70/2 = 35$. The answer is 35.

프롬프트 엔지니어링 기법

❖ 사고 트리(tree of thoughts, ToT, 2023) 프롬프팅

- 여러 문제해결 방법을 동시에 탐색해가며
각 경로의 장단점을 비교하면서 적합한 답변 선택

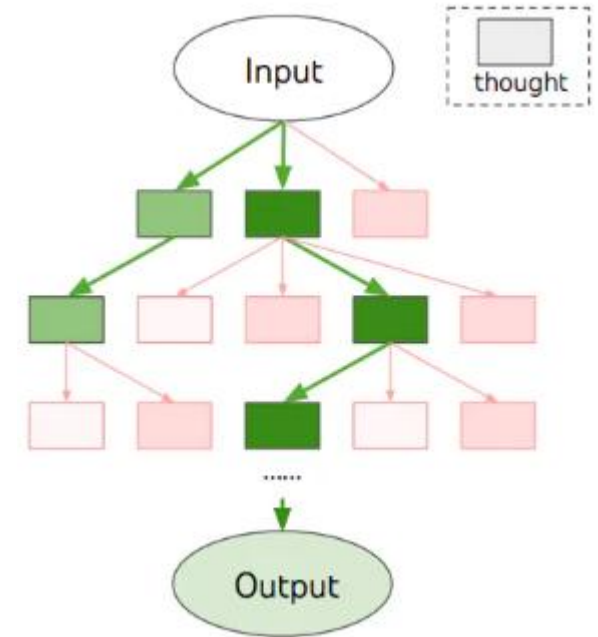
Imagine **three different experts** are answering this question.

All experts will write down 1 step of their thinking, then share it with the group.

Then all experts will go on to the next step, etc.

If any expert realizes they are wrong at any point then they leave.

The question is how to study generate ai in undergraduate school.



프롬프트 엔지니어링 기법

❖ 사고 그래프(graph of thoughts, 2023)

- 문제 해결 구조로 그래프 사용하는 방법

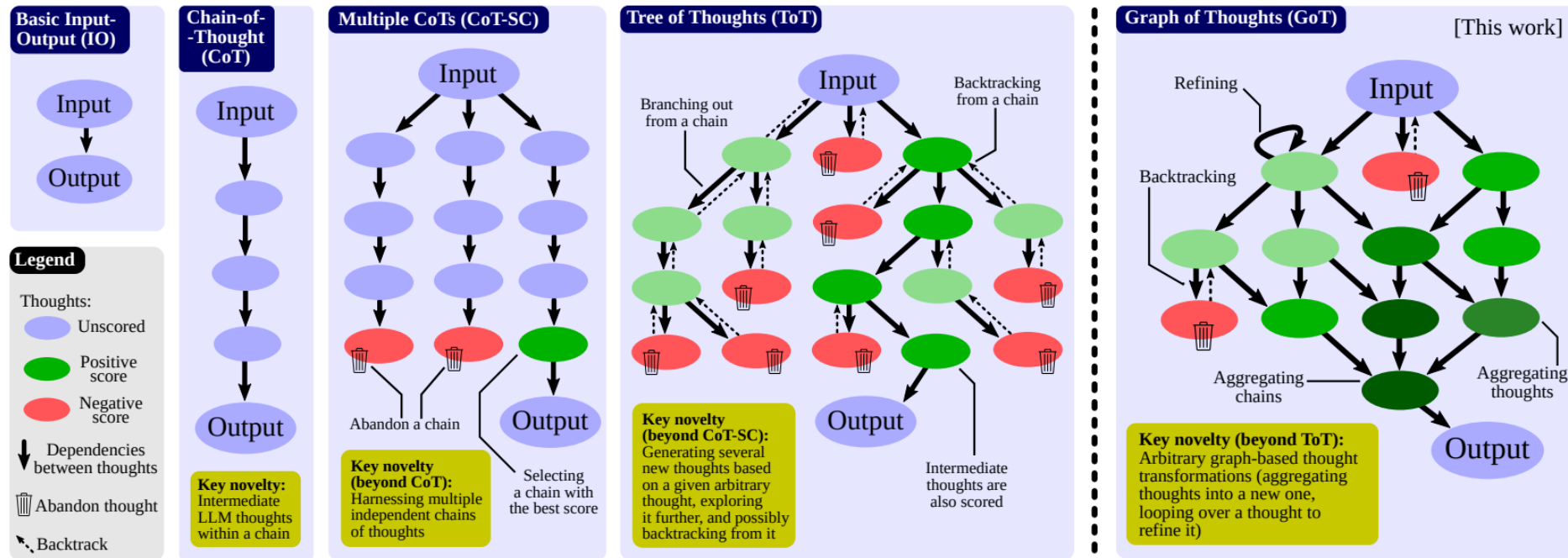


Figure 1: Comparison of Graph of Thoughts (GoT) to other prompting strategies.

프롬프트 엔지니어링 기법

❖ ReAct (2022)

- Reason + Act
- 주어진 입력에 대해 모델이 응답을 생성한 후, 그 응답을 스스로 평가하고 응답을 수정하거나, 추가적인 정보를 요청하거나, 다른 접근 방식을 시도하는 등의 행동 반복

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

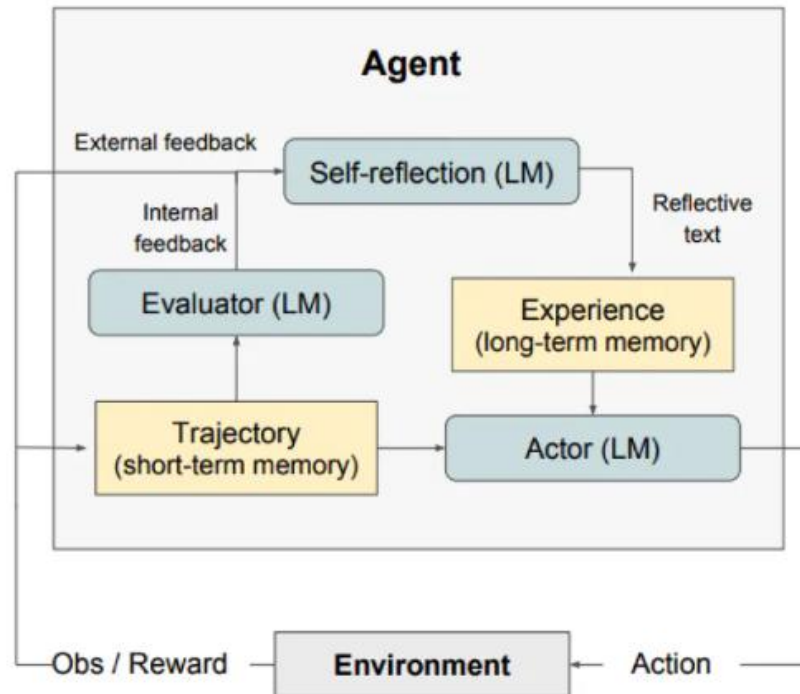
Act 4: Finish[keyboard function keys]



프롬프트 엔지니어링 기법

❖ Reflexion (2023)

- LLM 매개변수의 선택과 쌍을 이루는 에이전트의 메모리 인코딩으로 정책을 매개변수화하여 이루어 낸 '언어적' 강화 패러다임
- 환경 피드백(free-form language 또는 scalar, 자기 성찰)의 언어 피드백 변환
- 자기 평가(self-evaluation), 자기 성찰(self-reflection) 및 메모리 컴포넌트를 도입하여 ReAct 프레임워크 확장



Algorithm 1 Reinforcement via self-reflection

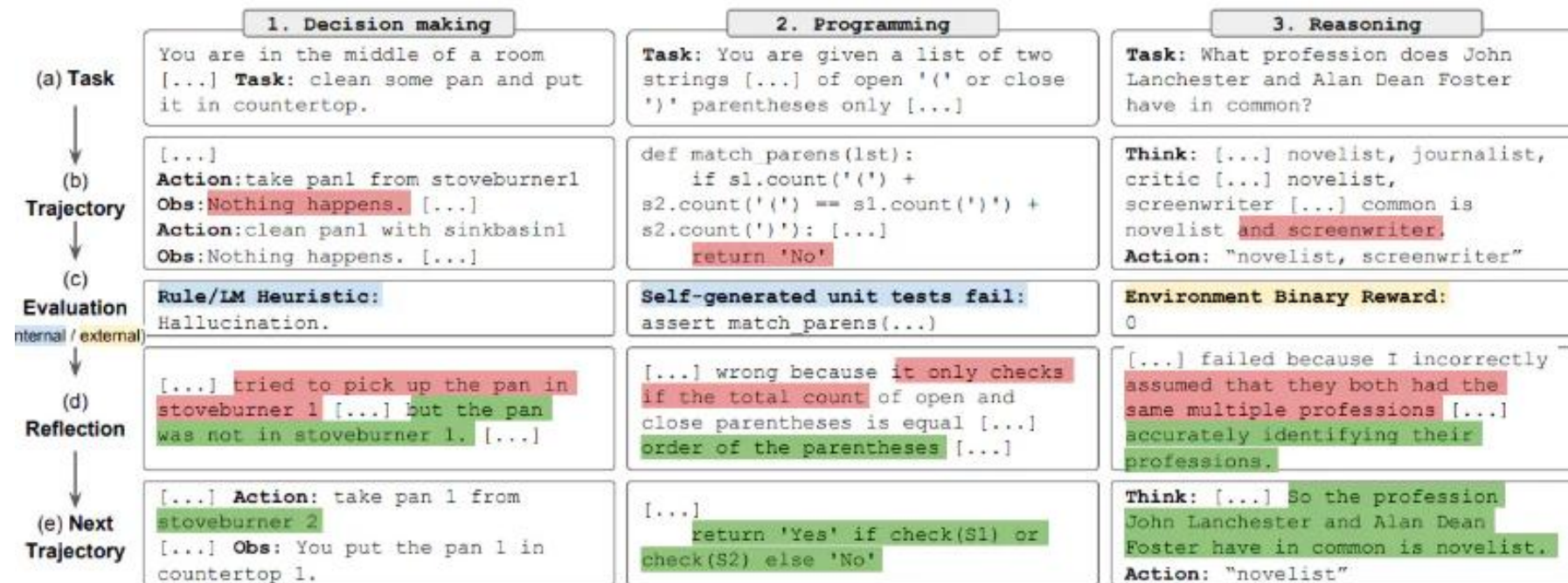
```
Initialize Actor, Evaluator, Self-Reflection:  
 $M_a, M_e, M_{sr}$   
Initialize policy  $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$   
Generate initial trajectory using  $\pi_\theta$   
Evaluate  $\tau_0$  using  $M_e$   
Generate initial self-reflection  $sr_0$  using  $M_{sr}$   
Set  $mem \leftarrow [sr_0]$   
Set  $t = 0$   
while  $M_e$  not pass or  $t < \text{max trials}$  do  
    Generate  $\tau_t = [a_0, o_0, \dots, a_i, o_i]$  using  $\pi_\theta$   
    Evaluate  $\tau_t$  using  $M_e$   
    Generate self-reflection  $sr_t$  using  $M_{sr}$   
    Append  $sr_t$  to  $mem$   
    Increment  $t$   
end while  
return
```

프롬프트 엔지니어링 기법

❖ Reflexion - cont.

▪ 주요 단계

- a) 작업 정의, b) 궤적 생성, c) 평가, d) 성찰(reflection) 수행, e) 다음 궤적 생성



Quiz

❖ AI 에이전트의 정의로 가장 적합한 것은?

- ① 데이터를 저장하는 시스템
- ② 환경을 인식하고 계획을 세우며 도구를 활용해 자율적으로 목표를 달성하는 소프트웨어
- ③ 단순히 텍스트를 생성하는 모델
- ④ 규칙 기반으로만 동작하는 프로그램

❖ AI 에이전트에서 LLM이 담당하는 역할은?

- ① 도구 실행
- ② Brain(지식기반 및 추론엔진)
- ③ 메모리 저장
- ④ 실행환경 관리

❖ Simple Reflex Agent의 동작 방식은?

- ① 목표를 세우고 행동
- ② 유틸리티 최대화
- ③ 조건(Condition)에 따라 즉각 행동
- ④ 모델 기반으로 상태 유지

Quiz

❖ Model-based Reflex Agent가 Simple Reflex Agent와 다른 점은?

- ① 도구를 사용할 수 있다
- ② 내부 상태(State)와 세계 모델을 유지한다
- ③ 멀티 에이전트 협력이 가능하다
- ④ LLM을 사용한다

❖ Goal-based Agent의 핵심 특징은?

- ① 조건-행동 규칙만 사용
- ② 유틸리티 함수로 최적화
- ③ 목표(Goals)를 바탕으로 행동 선택
- ④ 반사적으로만 동작

❖ Utility-based Agent가 Goal-based Agent와 다른 점은?

- ① 도구 사용 여부
- ② 유틸리티(행복도)를 통해 최적 행동 선택
- ③ 메모리 유무
- ④ 계획 수립 여부

Quiz

❖ 일반 LLM과 Agentic LLM의 차이점으로 틀린 것은?

- ① 일반 LLM은 단발성 응답, Agentic LLM은 반복 루프 수행
- ② Agentic LLM은 외부 도구 활용 가능
- ③ Agentic LLM은 목표를 능동적으로 추구
- ④ 일반 LLM도 API 호출이 자유롭다

❖ 에이전트의 단기(short-term) 메모리에 해당하는 것은?

- ① 외부 데이터베이스
- ② 현재 대화 컨텍스트(Context)
- ③ 학습 데이터
- ④ 파일 시스템

❖ 에이전트 구성요소 중 '계획수립(planning)'의 역할은?

- ① 외부 API 호출
- ② 복잡한 목표를 하위 태스크로 분해하고 실행 순서 결정
- ③ 메모리 저장
- ④ 도구 등록

Quiz

❖ 에이전트의 도구(tool)에 대한 설명으로 틀린 것은?

- ① 웹 검색, 코드 실행, API 호출 등이 포함된다
- ② JSON Schema로 정의해 호출된다
- ③ 도구 목록은 에이전트의 작업범위를 결정한다
- ④ 도구는 에이전트 내부에서만 실행된다

❖ 계획수립과 실행의 분리에서 Evaluator의 역할은?

- ① 계획 생성
- ② 계획 검증
- ③ 계획 실행
- ④ 도구 등록

❖ ReAct 추론 패턴에서 반복되는 두 가지 요소는?

- ① 계획(Plan)과 실행(Execute)
- ② 추론(Reason)과 행동(Act)
- ③ 기억(Memory)과 반성(Reflect)
- ④ 입력(Input)과 출력(Output)

Quiz

❖ Tree-of-Thoughts(ToT) 프롬프팅의 특징은?

- ① 단계별 순차 추론만 수행
- ② 여러 가설을 트리 형태로 탐색하며 비교
- ③ 단일 경로만 탐색
- ④ 계산 비용이 낮다

❖ Chain-of-Thought(CoT) 프롬프팅이 효과적인 문제 유형은?

- ① 복잡한 창의적 문제
- ② 단계별 순차 추론이 필요한 단순 문제
- ③ 게임 최적화 문제
- ④ 다중 에이전트 협력

❖ 에이전트 실행환경(Runtime)의 주요 기능이 아닌 것은?

- ① LLM 호출 관리
- ② 도구 실행 관리
- ③ 에이전트 루프 제어
- ④ 사용자 인터페이스 디자인

Quiz

❖ 로컬 환경에서 에이전트를 실행할 때의 장점이 아닌 것은?

- ① 사용자 이메일, 로컬 파일에 직접 접근 가능
- ② 민감한 정보 보호에 유리
- ③ 무한한 확장성
- ④ 장기 작업 실행 가능

❖ MCP를 USB-C에 비유하는 이유는?

- ① 속도가 빠르기 때문에
- ② 다양한 AI 모델과 외부 시스템 간 공통 인터페이스 역할이기 때문에
- ③ 무선 통신을 지원하기 때문에
- ④ 전력을 공급하기 때문에

❖ MCP Host에 해당하지 않는 것은?

- ① Claude Desktop
- ② Cursor
- ③ Weather API 서버
- ④ 자체 LLM 앱

Quiz

❖ MCP 통신 프로토콜로 사용되는 것은?

- ① REST API
- ② GraphQL
- ③ JSON-RPC 2.0
- ④ gRPC

❖ MCP의 SSE(Server-Sent Events) 방식의 특징으로 틀린 것은?

- ① HTTP 기반 푸시 방식
- ② URL을 통해 접근
- ③ 클라이언트 시작 시 한 번에 spawn하여 세션 내내 상주
- ④ 호출 전에 서버가 먼저 실행되어야 함

❖ MCP Stdio 방식의 통신 채널은?

- ① HTTP 엔드포인트
- ② WebSocket
- ③ 표준 입력/출력(stdin/stdout) 파이프
- ④ UDP 소켓

Quiz

❖ A2A Agent Card에 포함되지 않는 정보는?

- ① 에이전트 이름(name)과 설명(description)
- ② 보안 요구사항(security requirements)
- ③ 에이전트의 학습 데이터셋
- ④ 통신 엔드포인트(communication endpoints)

❖ MCP와 A2A의 차이를 설명한 것으로 올바른 것은?

- ① MCP는 AI끼리 수평 연결, A2A는 AI-도구 수직 연결
- ② MCP는 AI-도구/데이터 수직 연결, A2A는 AI-AI 에이전트 수평 연결
- ③ 둘 다 동일한 역할이다
- ④ MCP는 구글, A2A는 Anthropic이 만들었다

❖ LangChain의 RAG 워크플로우에서 Splitter의 역할은?

- ① 외부 데이터 적재
- ② 데이터를 적당한 크기로 분할/결합
- ③ 임베딩 벡터 생성
- ④ LLM 응답 생성

Quiz

❖ LangGraph에서 '노드(Node)'는 무엇을 의미하는가?

- ① LLM 에이전트나 도구
- ② 노드 간 통신 채널
- ③ 상태 저장소
- ④ 사용자 인터페이스

❖ LangGraph의 특징으로 틀린 것은?

- ① 상태 유지 가능
- ② 그래프 구조 사용
- ③ 각 작업 후 자동으로 상태 저장
- ④ 단일 에이전트만 지원

❖ 프롬프트 엔지니어링의 정의로 적합한 것은?

- ① LLM을 학습시키는 기법
- ② 언어모델을 효과적으로 활용하기 위한 프롬프트의 개발 및 최적화
- ③ 신경망 구조를 설계하는 방법
- ④ 데이터를 전처리하는 기술

Quiz

❖ 프롬프트의 구성요소가 아닌 것은?

- ① 지시(instruction)
- ② 문맥(context)
- ③ 가중치(weight)
- ④ 출력 지시자(output indicator)

❖ 프롬프트 설계 시 '할 것(to do)'을 지시하는 것이 '하지 말 것(not to do)'보다 나은 이유는?

- ① 더 짧기 때문에
- ② 모델이 긍정적 지시에 더 잘 반응해 원하는 행동을 유도할 수 있기 때문에
- ③ 문법적으로 더 정확하기 때문에
- ④ API 비용이 절감되기 때문에

❖ Zero-shot Prompting이란?

- ① 많은 예시를 제공하는 프롬프팅
- ② 예시 없이 직접 질문/지시하는 프롬프팅
- ③ 단계별 추론을 유도하는 프롬프팅
- ④ 여러 답변 중 최선을 선택하는 방법

Quiz

❖ Few-shot Prompting에서 예시를 제공하는 목적은?

- ① 모델 파라미터 업데이트
- ② In-context learning으로 답변 방향과 형식을 유도하기 위해
- ③ API 오류 방지
- ④ 토큰 수 절감

❖ Zero-shot CoT에서 사용하는 'trigger sentence'는?

- ① Think about it carefully.
- ② Let's think step by step.
- ③ The answer is obvious.
- ④ Please be precise.

❖ Self-Consistency 기법의 핵심 아이디어는?

- ① 프롬프트를 반복 수정
- ② 동일 프롬프트로 여러 결과를 생성하고 다수결(투표)로 최선 선택
- ③ 단일 응답만 사용
- ④ 외부 검색 결과를 활용

Quiz

❖ **Tree of Thoughts(ToT) 프롬프팅에서 '전문가 3명이 답변한다'는 표현의 의미는?**

- ① 실제 전문가를 고용하는 것
- ② 여러 사고 경로를 병렬로 탐색하여 비교하는 것
- ③ 멀티 에이전트 시스템을 구성하는 것
- ④ 투표로 최선 답변 선택

❖ **Graph of Thoughts(GoT)가 Tree of Thoughts(ToT)를 넘어서는 핵심 기능은?**

- ① 더 짧은 응답 생성
- ② 임의 그래프 기반 사고 변환(집계, 루프, 역추적)
- ③ 계산 비용 절감
- ④ API 없이 동작

❖ **ReAct 프롬프팅의 동작 흐름으로 올바른 것은?**

- ① Input → Output
- ② Thought → Action → Observation → Thought ... → Answer
- ③ Plan → Execute → Reflect
- ④ Query → Planner → Evaluator → Executor

Quiz

❖ Reflexion 프레임워크가 ReAct를 확장한 방식은?

- ① 더 많은 도구 추가
- ② 자기 평가(self-evaluation), 자기 성찰(self-reflection), 메모리 컴포넌트 추가
- ③ 멀티 에이전트 지원
- ④ CoT 기법 통합

❖ Reflexion에서 '궤적(trajecory)'이 저장되는 메모리 유형은?

- ① 장기 메모리(Experience)
- ② 단기 메모리(Trajectory)
- ③ 외부 데이터베이스
- ④ 학습 파라미터

❖ 에이전트 계획 형태 중 병렬(Parallel) 실행이 적합한 예는?

- ① 텍스트를 SQL로 변환 후 실행
- ② 베스트셀러 제품 찾은 후 각 항목의 가격 병렬 조회
- ③ 수익 보고서 분석 후 매수/매도 결정
- ④ 바이럴 포스트 원본 추적(for loop)

Quiz

❖ Few-shot CoT와 Zero-shot CoT의 차이는?

- ① 사용 모델의 차이
- ② 사고 과정 예시 제공 여부
- ③ 도구 사용 여부
- ④ 출력 형식의 차이

❖ 프롬프트 설계에서 '애매한 표현 회피'의 올바른 예시는?

- ① '설명을 간략하게 해주세요'
- ② '고등학생에게 2-3 문장으로 프롬프트 엔지니어링 개념을 설명하세요'
- ③ '자세하지 않게 설명해주세요'
- ④ '적당한 길이로 작성해주세요'

❖ 다중 에이전트 시스템(Multi-Agent System)의 특징은?

- ① 하나의 에이전트만 존재
- ② 복수의 상호작용하는 에이전트로 구성되며 환경을 공유
- ③ LLM 없이 동작
- ④ 외부 도구 사용 불가

Quiz

❖ 에이전트 실행환경의 '라우팅(routing)' 기능이란?

- ① 외부 API 연결
- ② 입력 또는 상태에 따라 적절한 에이전트나 도구로 처리 흐름을 분기하는 것
- ③ 메모리 관리
- ④ 오류 복구

❖ LangGraph에서 '에지(Edge)'의 역할은?

- ① LLM 에이전트 정의
- ② 노드 간의 통신 채널 및 전환 조건 정의
- ③ 상태 저장
- ④ 도구 호출

❖ 프롬프트 엔지니어링에서 복잡한 작업을 수행할 때 권장하는 방법은?

- ① 한 번에 모든 것을 요청
- ② 수행할 과정을 번호로 구체적으로 단계별 기술
- ③ 최대한 짧게 요청
- ④ 영어로만 요청